香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 19

Modern computer systems: memory

Prof. Yunming XIAO

School of Data Science

Table of contents

- Expose semantics to manage far memory better
  - **AIFM**: High-Performance Applications-Integrated Far Memory, **OSDI'20**
- Reinvent paging for new workloads:
  - Efficient Memory Management for Large Language Model Serving with **PagedAttention**, **SOSP'23**
- Predict accesses before demand misses happen
  - **PF-LLM**: Large Language Model Hinted Hardware Prefetching, **ASPLOS'26 (Best Paper)**

Table of contents

- Expose semantics to manage far memory better
  - **AIFM**: High-Performance Applications-Integrated Far Memory, **OSDI'20**
- Reinvent paging for new workloads:
  - Efficient Memory Management for Large Language Model Serving with **PagedAttention**, **SOSP'23**
- Predict accesses before demand misses happen
  - **PF-LLM**: Large Language Model Hinted Hardware Prefetching, **ASPLOS'26 (Best Paper)**

AIFM: High-Performance, Application-Integrated Far Memory

Zain (Zhenyuan) Ruan* Malte Schwarzkopf† Marcos K. Aguilera ‡ Adam Belay*

*MIT CSAIL †Brown University ‡VMware Research

In-Memory Applications

Data Analytics

Web Caching

Database

Graph Processing

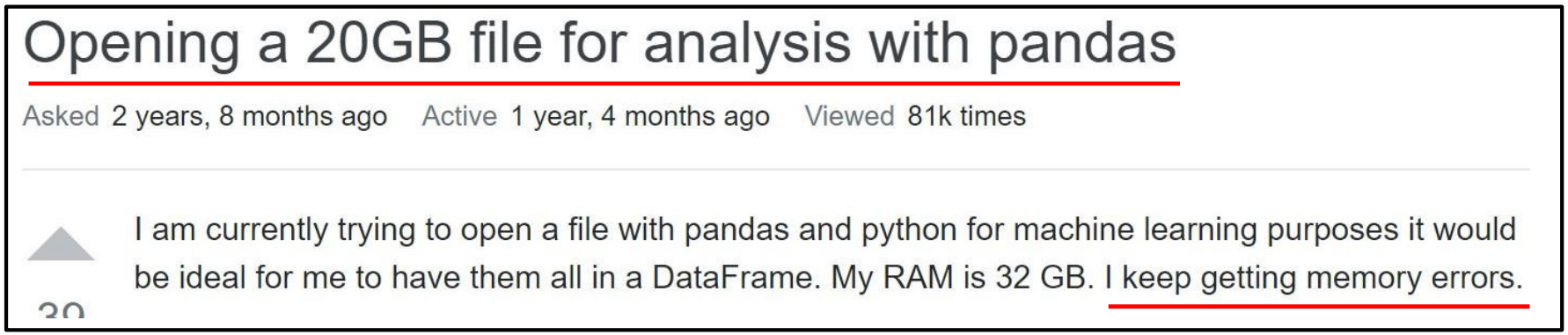
Memory Is Inelastic

➢Limited by the server physical boundary.

Memory Is Inelastic

- Limited by the server physical boundary.

➢Applications cannot overcommit



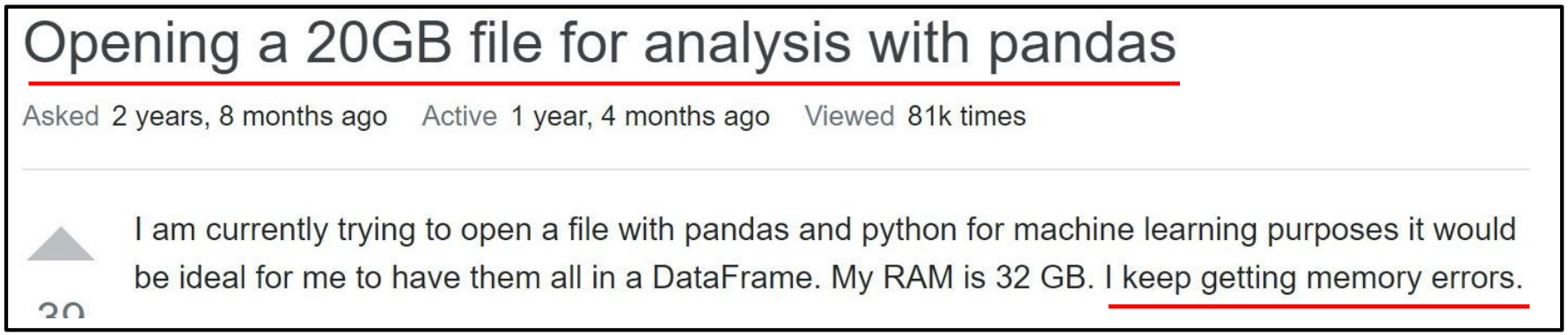
Opening a 20GB file for analysis with pandas

Asked 2 years, 8 months ago Active 1 year, 4 months ago Viewed 81k times

I am currently trying to open a file with pandas and python for machine learning purposes it would be ideal for me to have them all in a DataFrame. My RAM is 32 GB. I keep getting memory errors.

Memory Is Inelastic

- Limited by the server physical boundary.
- Applications cannot overcommit memory.



Opening a 20GB file for analysis with pandas

Asked 2 years, 8 months ago Active 1 year, 4 months ago Viewed 81k times

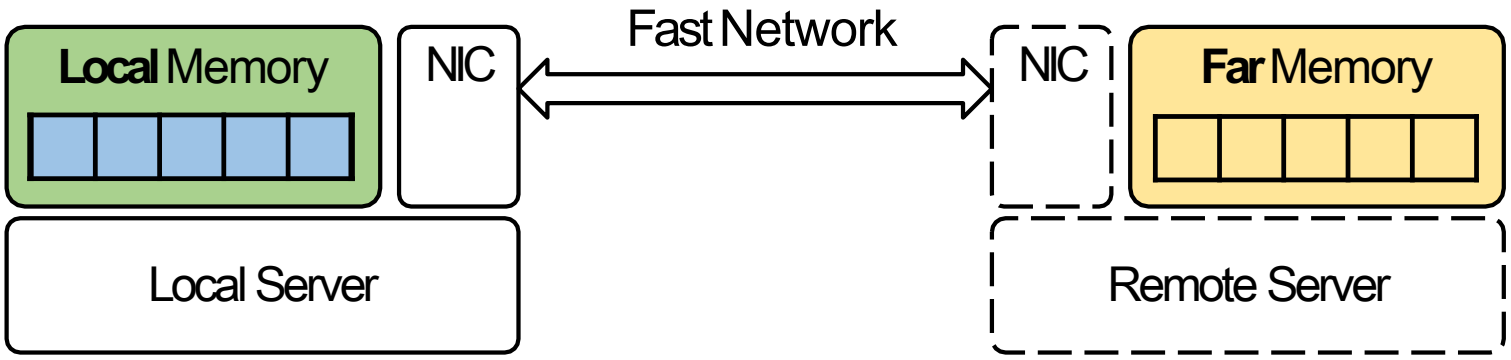
I am currently trying to open a file with pandas and python for machine learning purposes it would be ideal for me to have them all in a DataFrame. My RAM is 32 GB. I keep getting memory errors.

30

➢ Expensive solution: overprovision memory for peak usage.

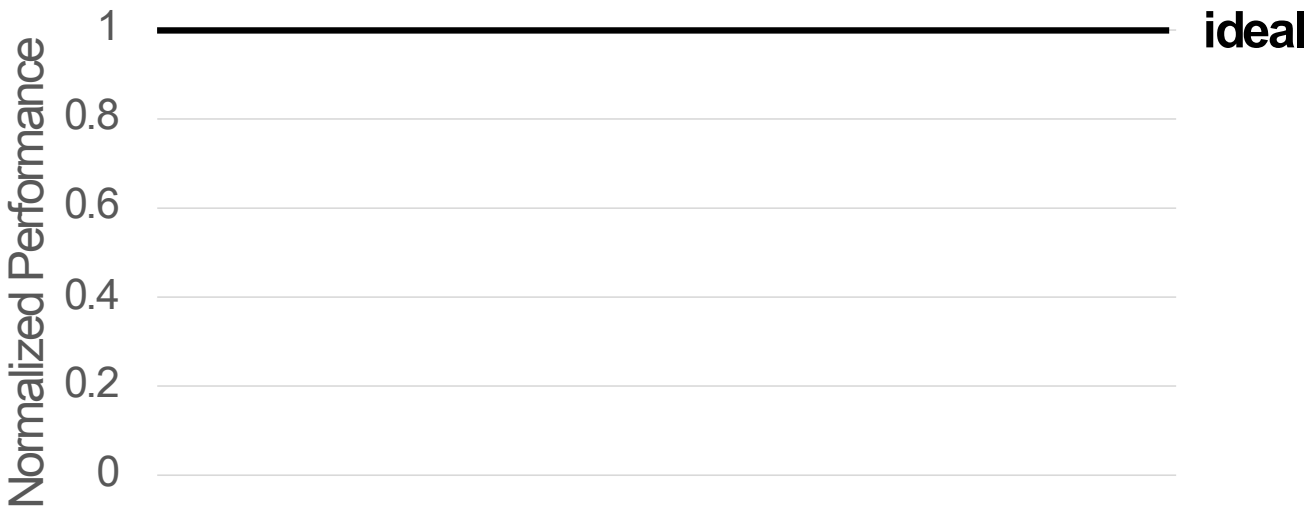
Trending Solution: Far Memory

➢ Leverage the idle memory of remote servers (with fast network).



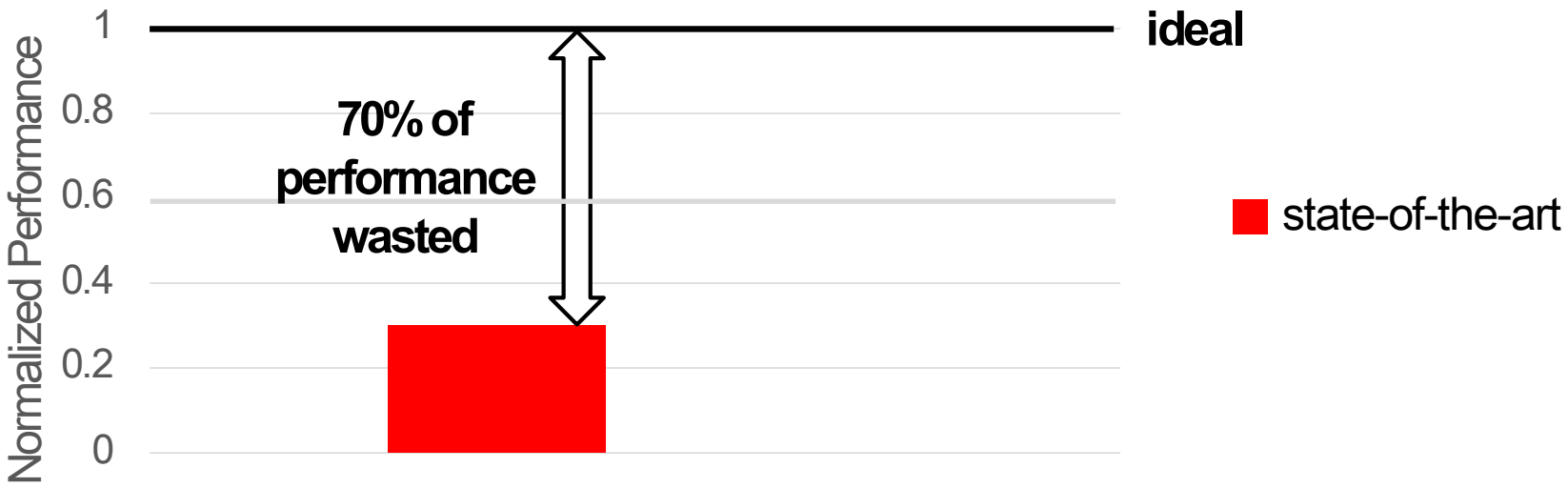
Existing Far-Memory Systems Perform Poorly

➢ Real-world Data Analytics from Kaggle.



Existing Far-Memory Systems Perform Poorly

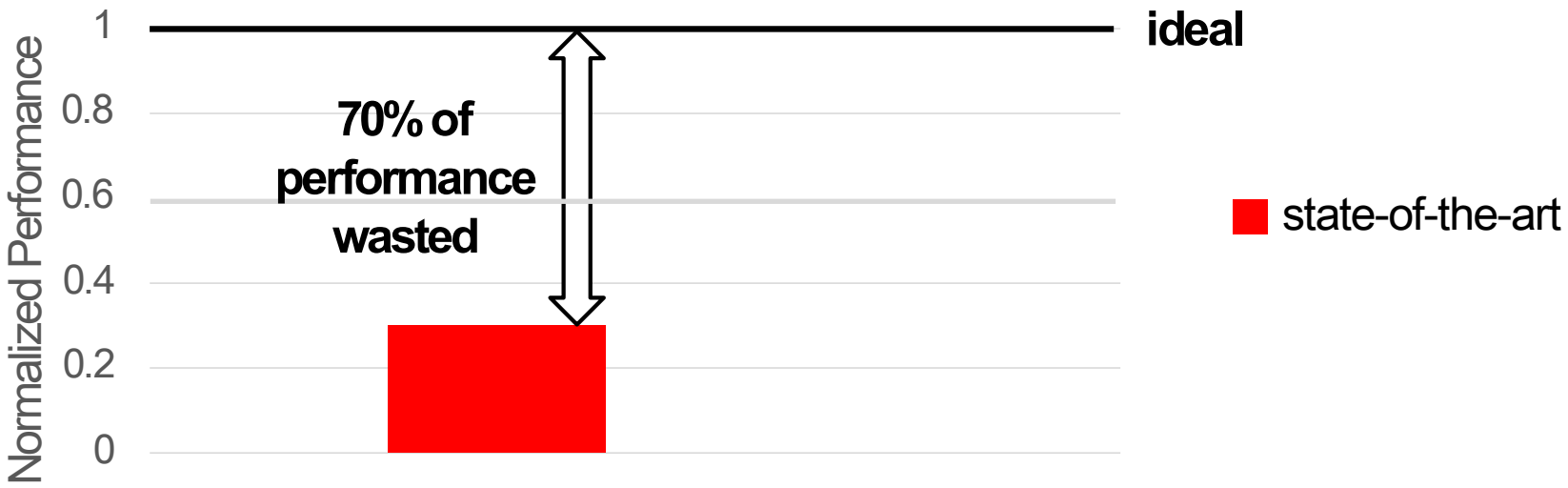
- Real-world Data Analytics from Kaggle.
  - ➢ Provision **25%** of working set in local mem



Existing Far-Memory Systems Perform Poorly

- Real-world Data Analytics from Kaggle.
  - Provision **25%** of working set in local mem.

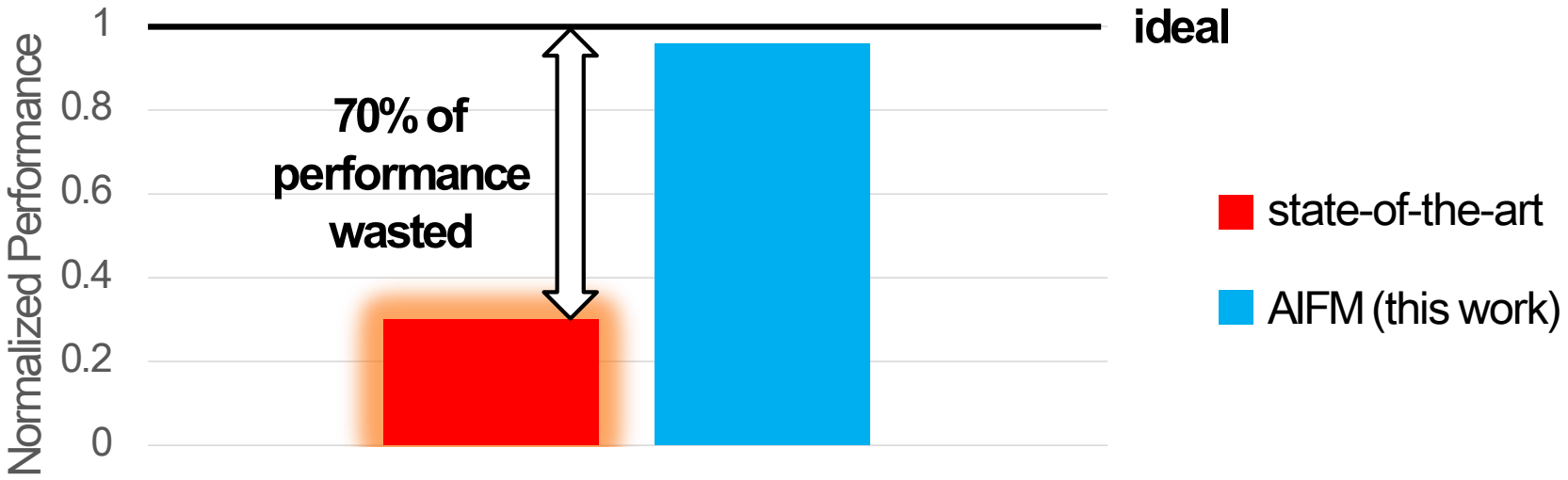
➢ Goal: reclaim the wasted performance.



Existing Far-Memory Systems Perform Poorly

- Real-world Data Analytics from Kaggle.
  - Provision **25%** of working set in local mem.

➢ Goal: reclaim the wasted performance.

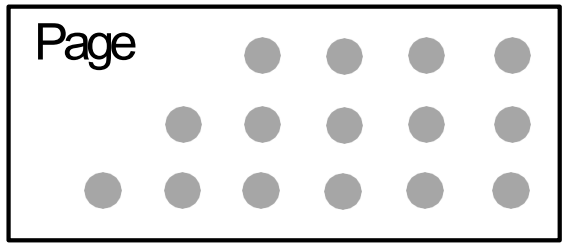


Why Do Existing Systems Waste Performance?

- Problem: based on **OS paging**.
  - Semantic gap.
  - High kernel overheads.

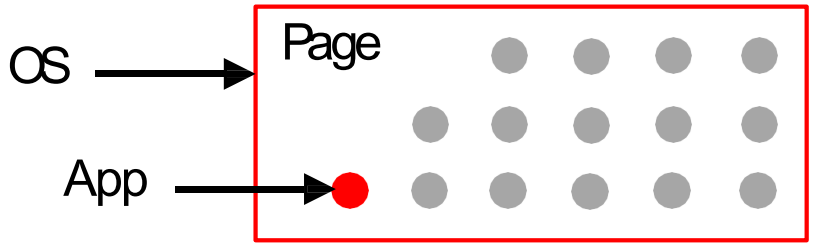
Challenge 1: Semantic Gap

➢ Page granularity → **R/W amplification.**



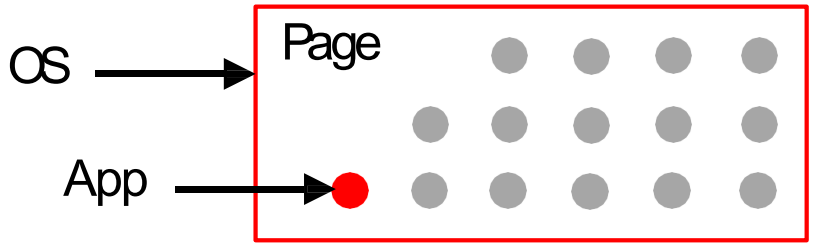
Challenge 1: Semantic Gap

➢ Page granularity → **R/W amplification.**



Challenge 1: Semantic Gap

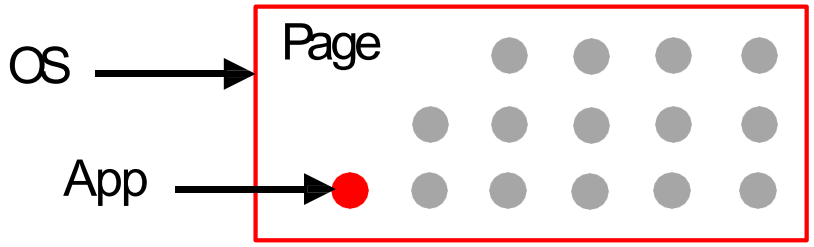
- Page granularity → **R/W amplification.**



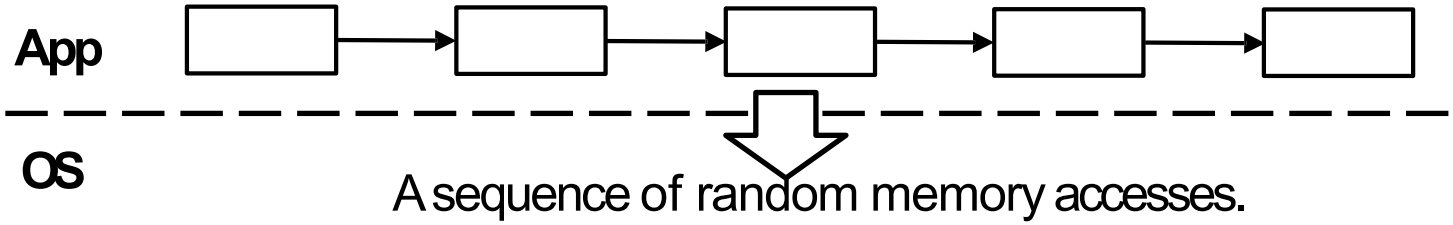
➢ OS lacks app knowledge → **hard to prefetch, etc.**

Challenge 1: Semantic Gap

- Page granularity → **R/W amplification.**

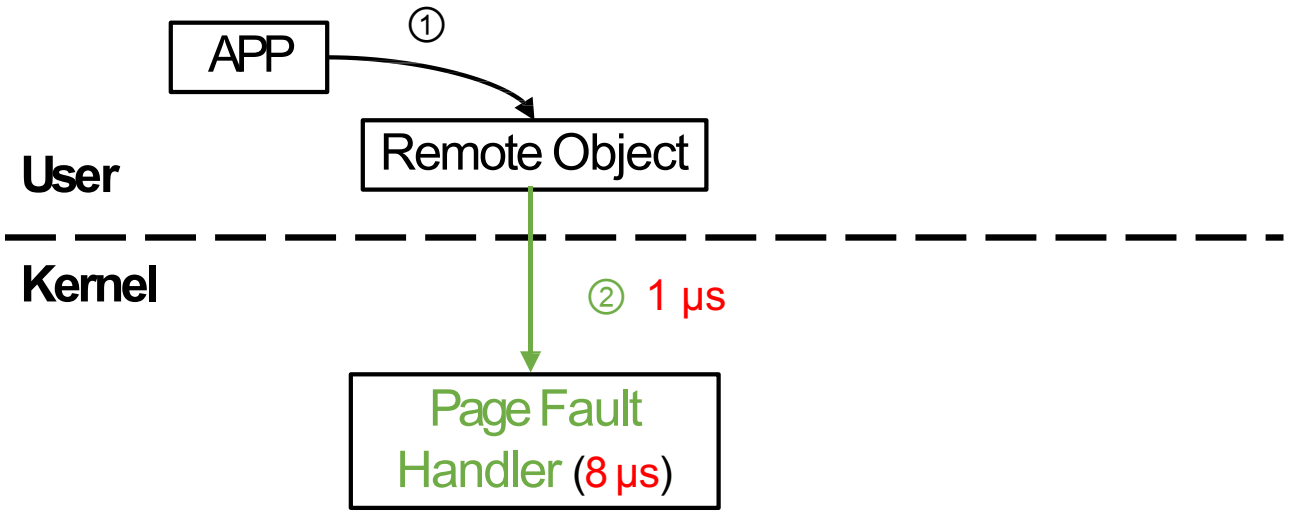


➢ OS lacks app knowledge → **hard to prefetch, etc.**



Challenge 2: High Kernel Overheads

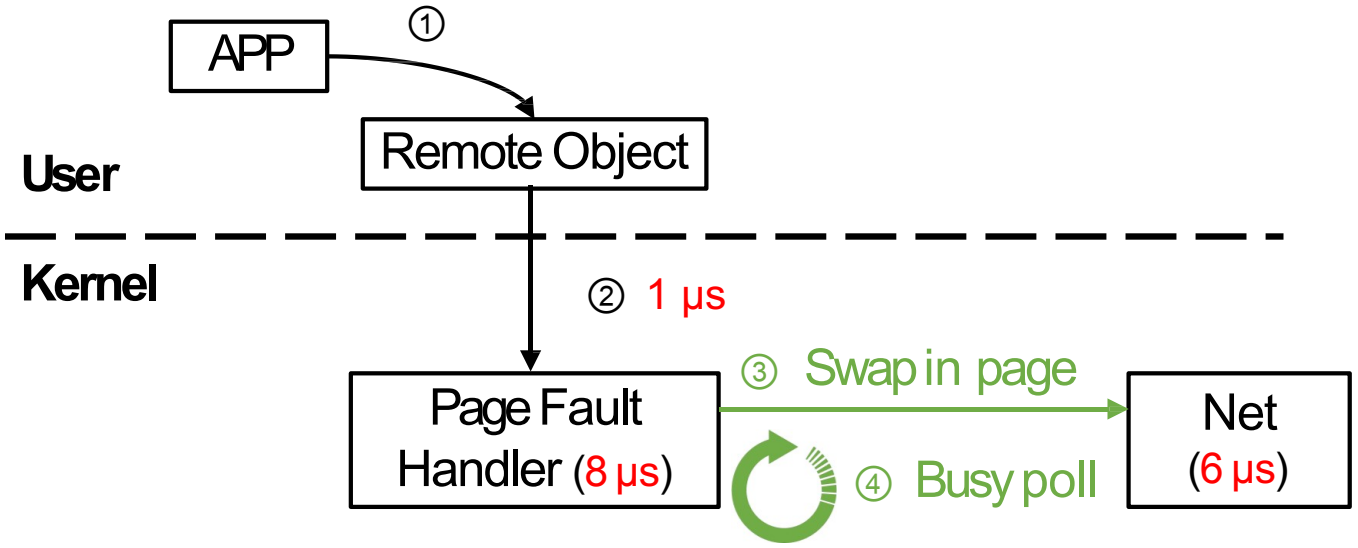
➢ **Expensive page faults.**



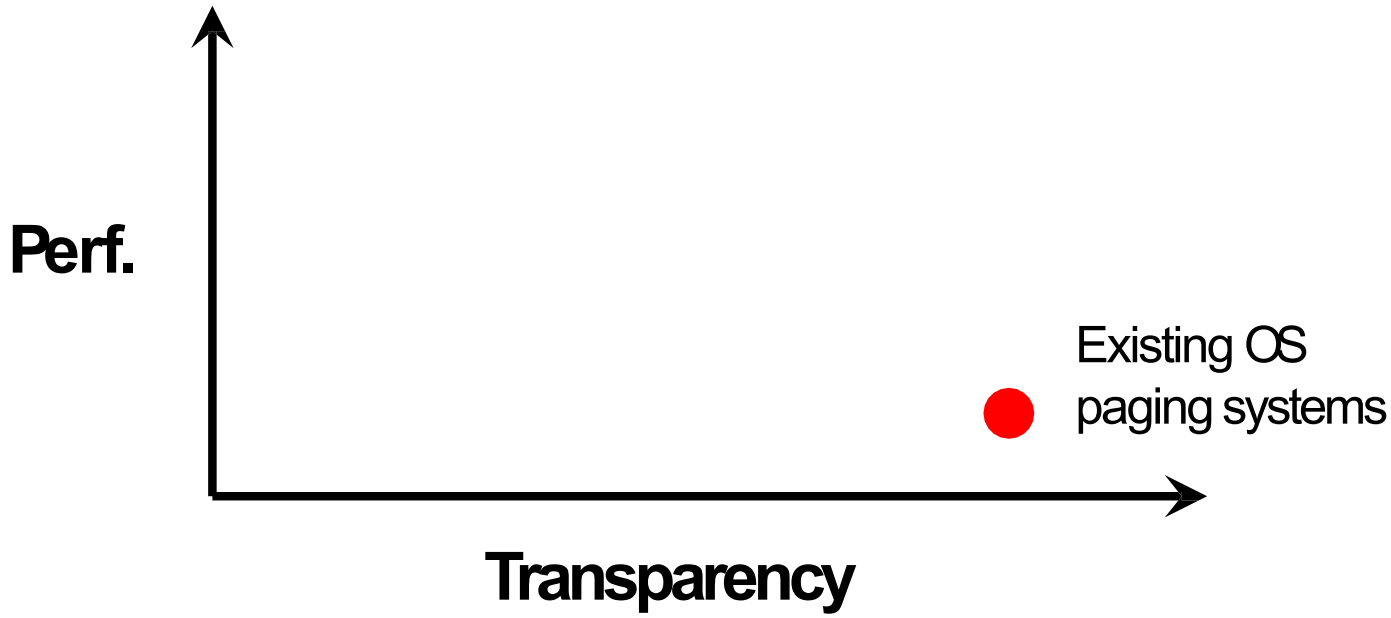
Challenge 2: High Kernel Overheads

- **Expensive page faults.**

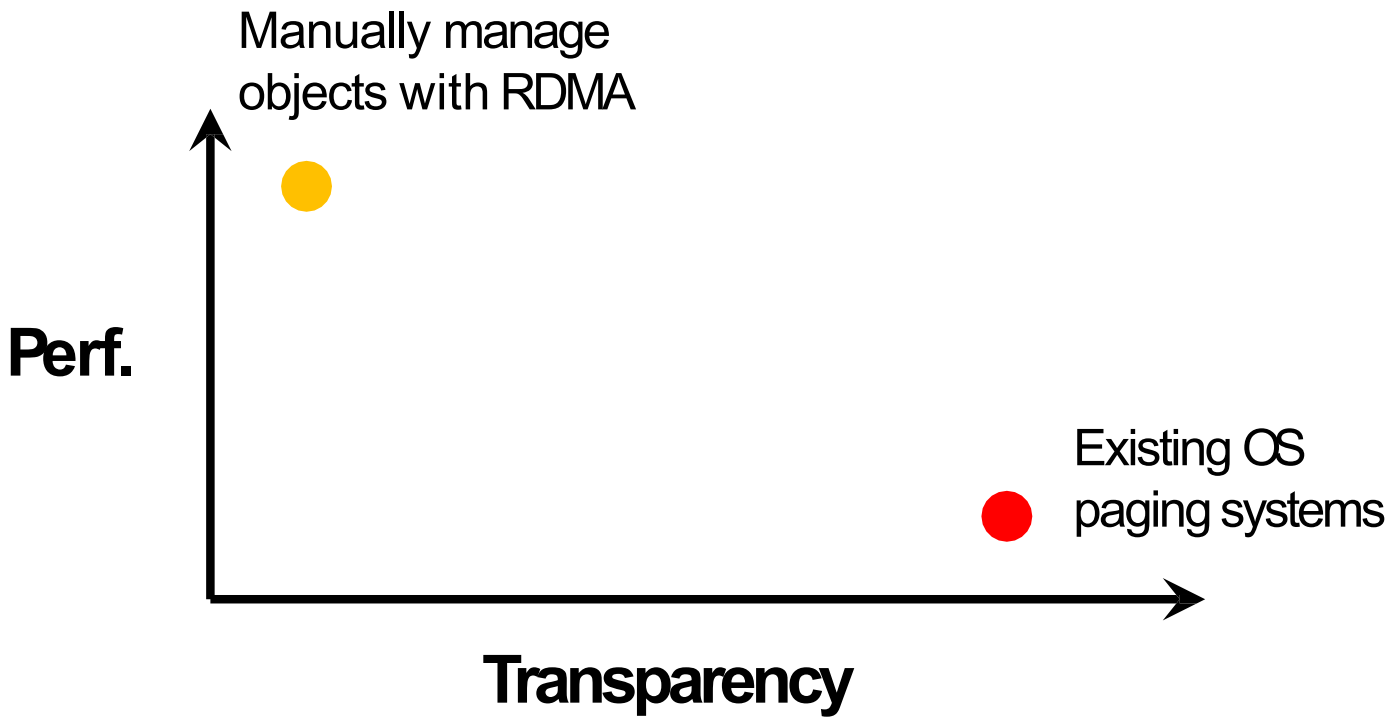
➢ Busy Polling for in-kernel net I/O → **burn CPU cycles**.



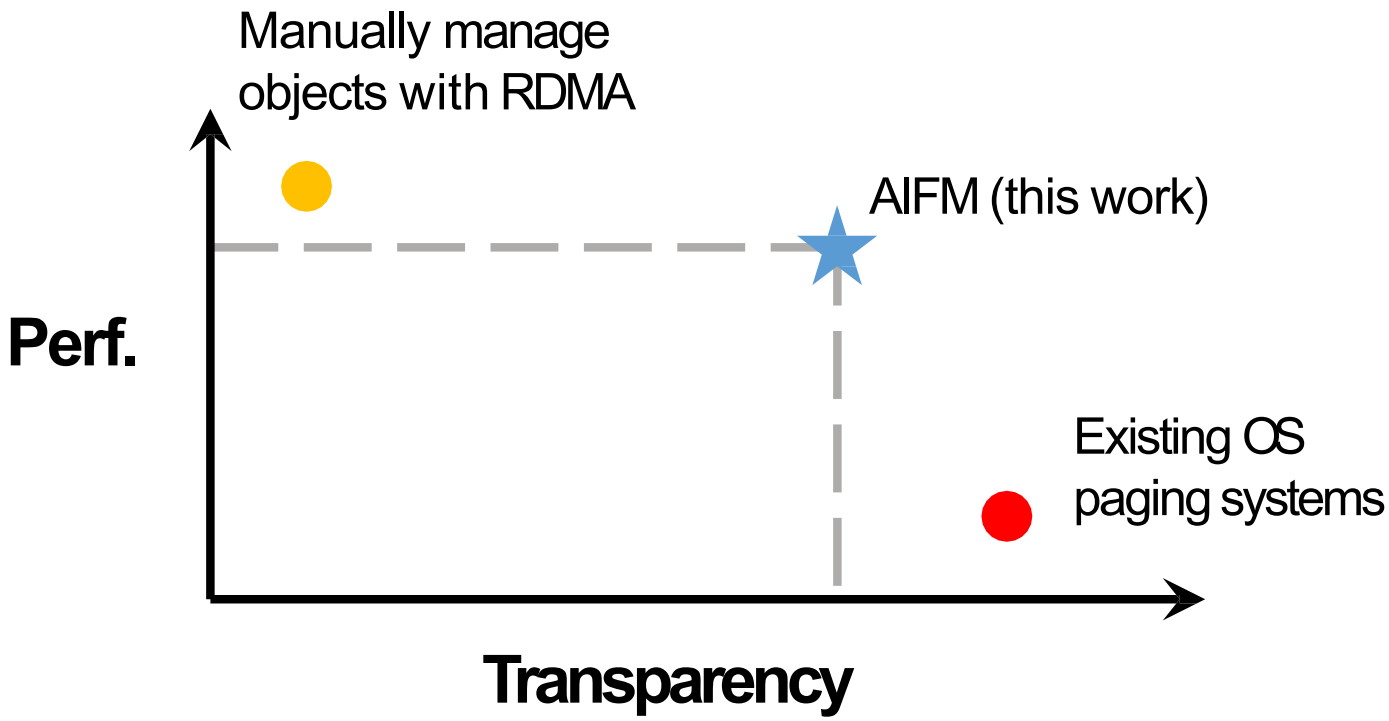
Design Space



Design Space



Design Space



AIFM's Design Overview

➢ Key idea: swap memory using a userspace runtime.

AIFM's Design Overview

➢Key idea: swap memory using a userspace runtime.

Challenge	Solution
1. Semantic gap (Amplification, Hard to prefetch)	Remoteable Data structure library
2. Kernel overheads (page faults, busy poll for net I/O)	Userspace runtime
3. Impact of Memory Reclamation (pause app threads)	Pauseless evacuator
4. Network BW < DRAM BW	Remote Agent

AIFM in Action

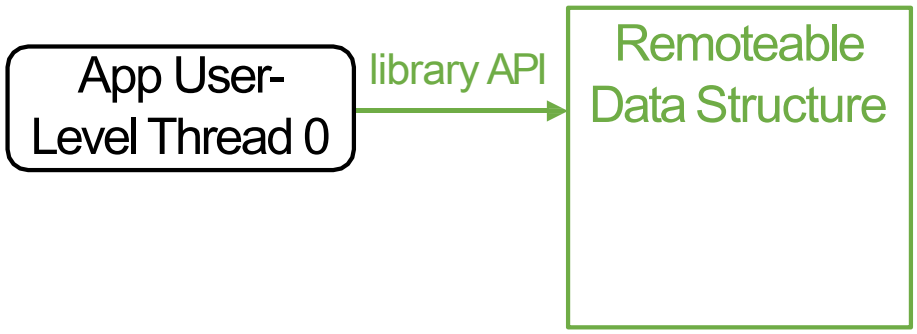
App User-Level Thread 0

Local Memory

Far Memory

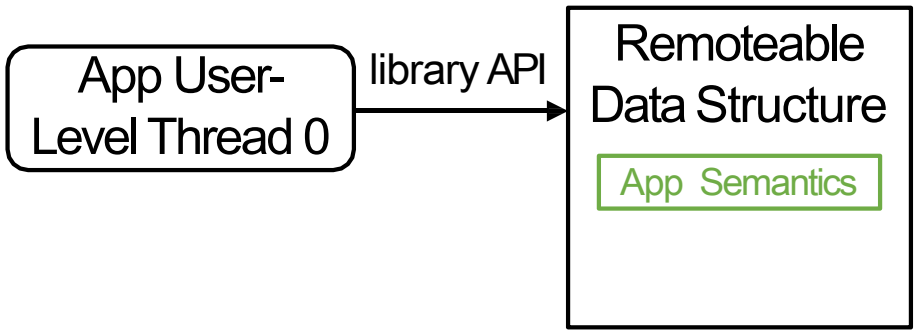
1. Remoteable Data Structure Library

➢ Solved challenge: semantic gap.



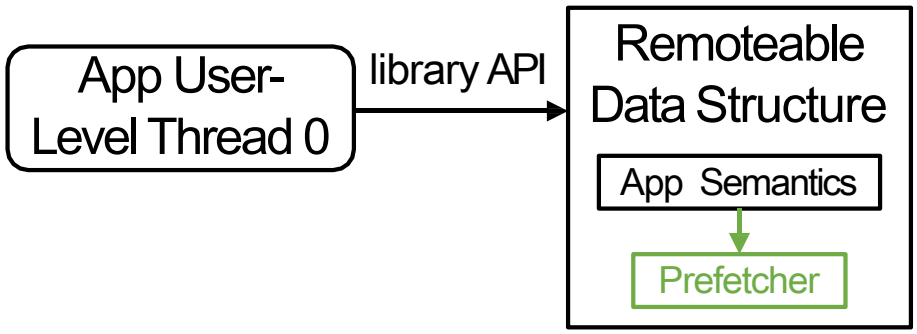
1. Remoteable Data Structure Library

➢ Solved challenge: semantic gap.



1. Remoteable Data Structure Library

➢ Solved challenge: semantic gap.

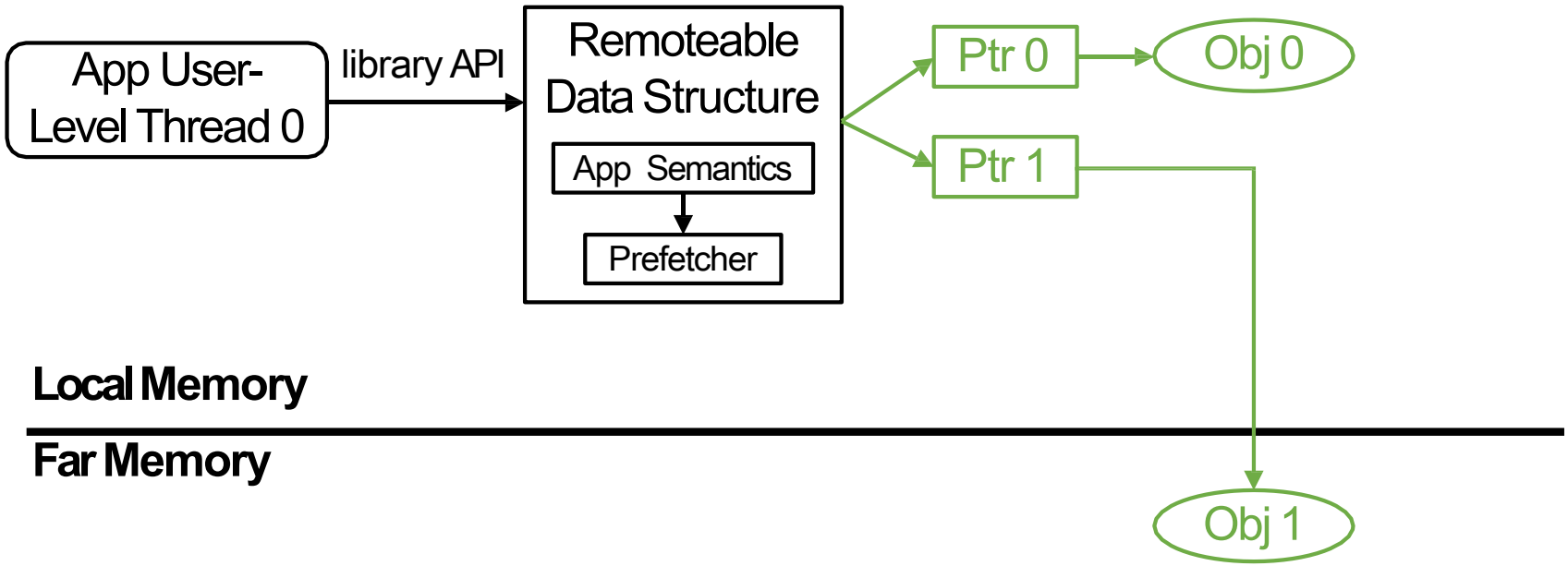


Local Memory

Far Memory

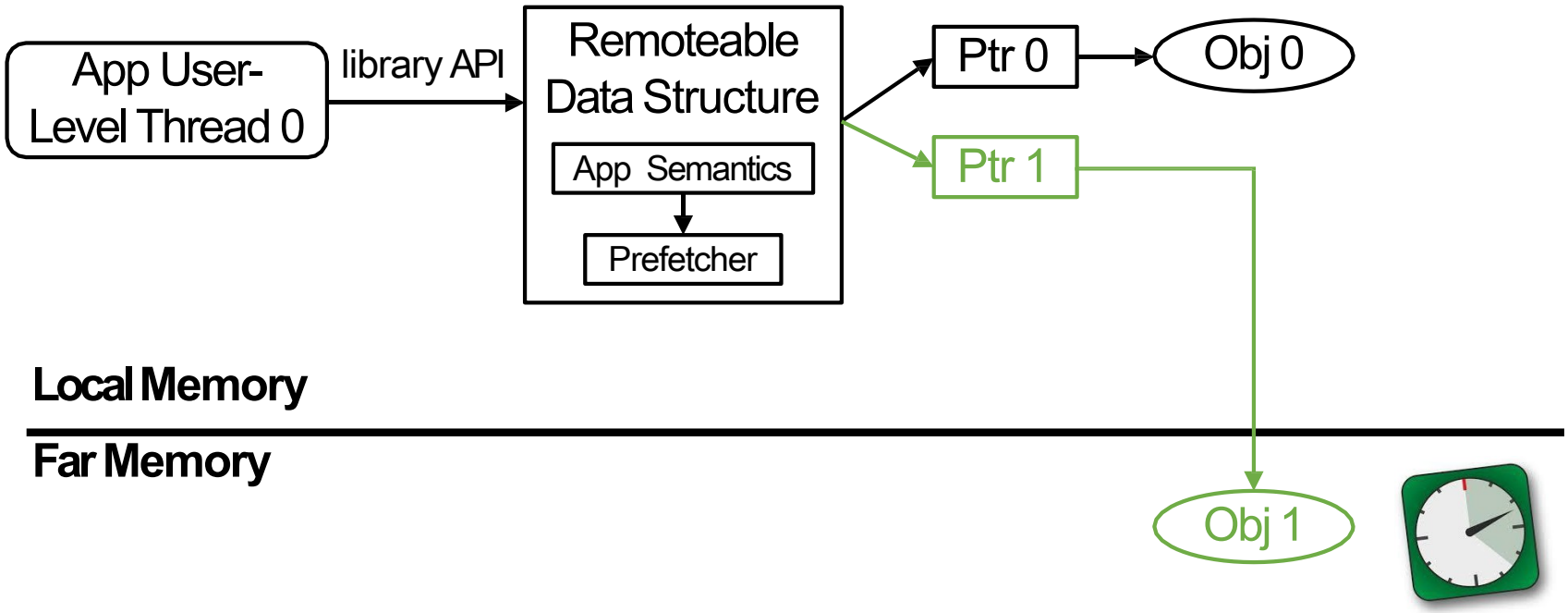
1. Remoteable Data Structure Library

➢ Solved challenge: semantic gap.



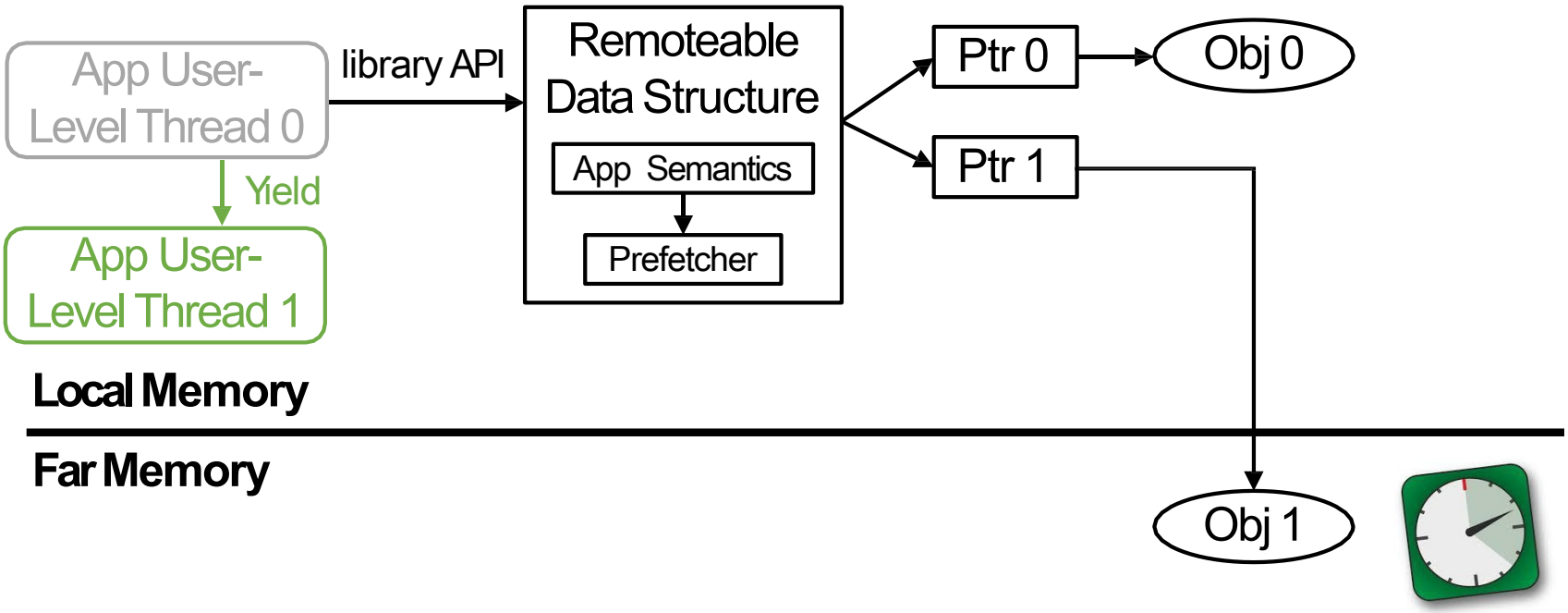
2. Userspace Runtime

➢ Solved challenge: kernel overheads.



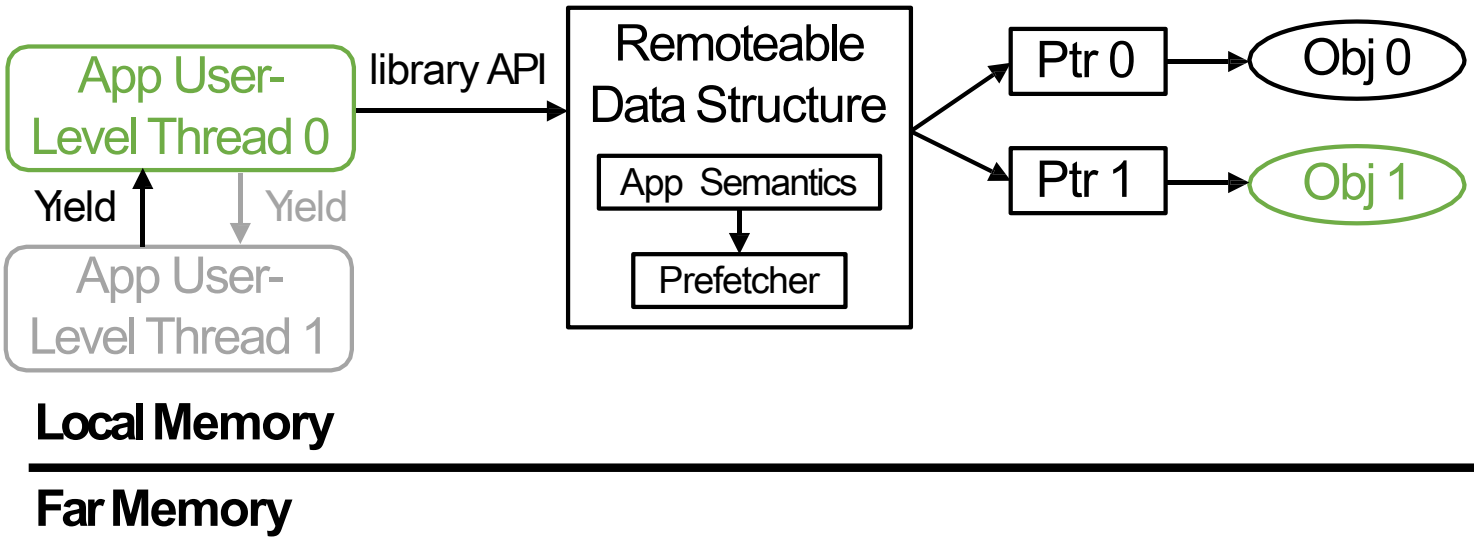
2. Userspace Runtime

➢ Solved challenge: kernel overheads.



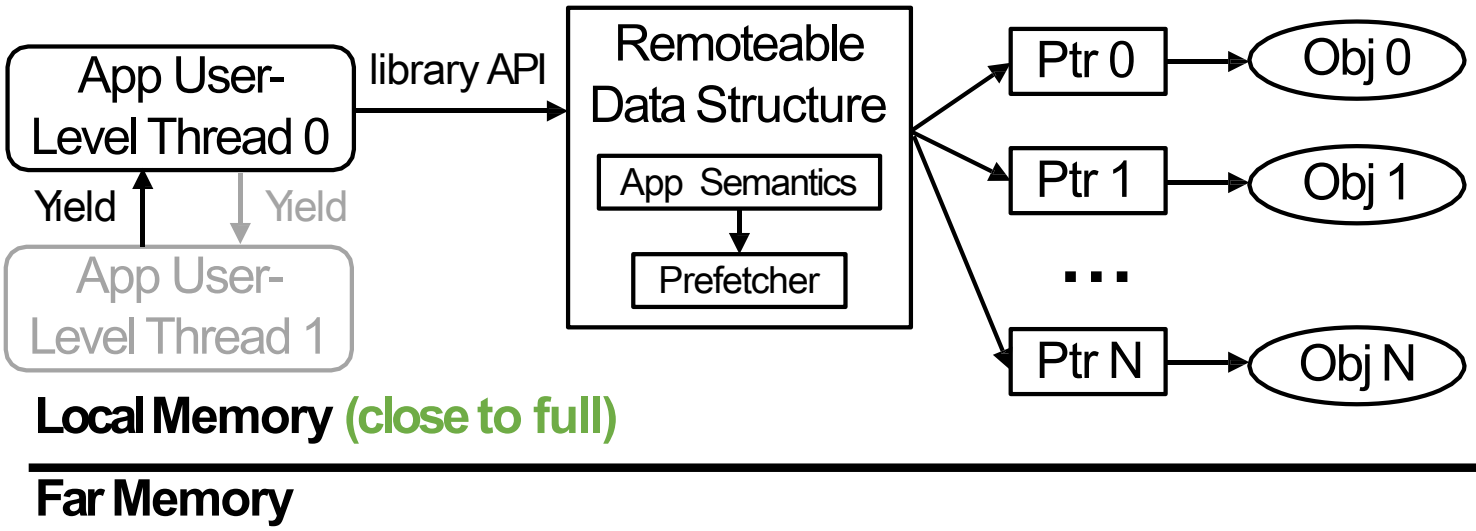
2. Userspace Runtime

➢ Solved challenge: kernel overheads.



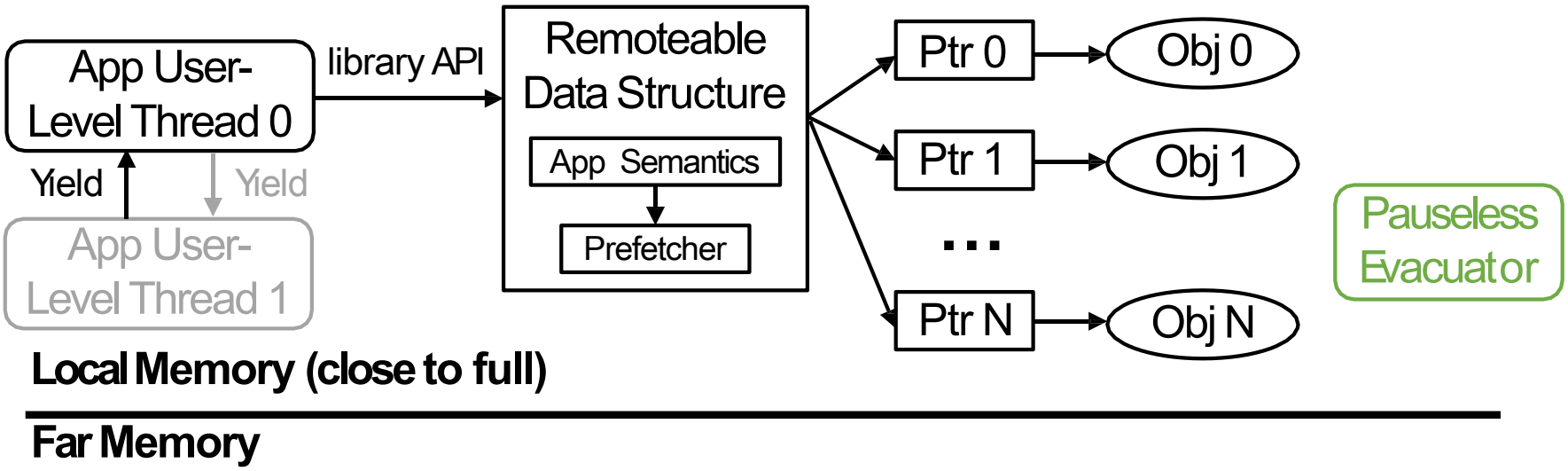
3. Pauseless Evacuator

➢ Solved challenge: impact of memory reclamation.



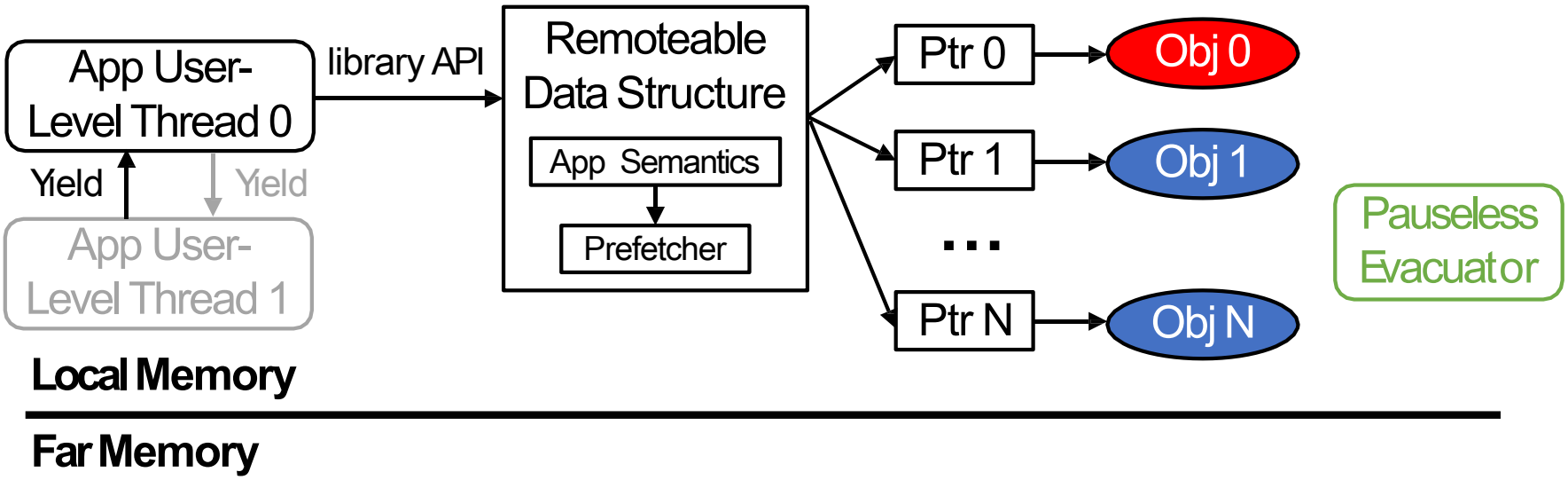
3. Pauseless Evacuator

➢Solved challenge: performance impact of memory reclamation.



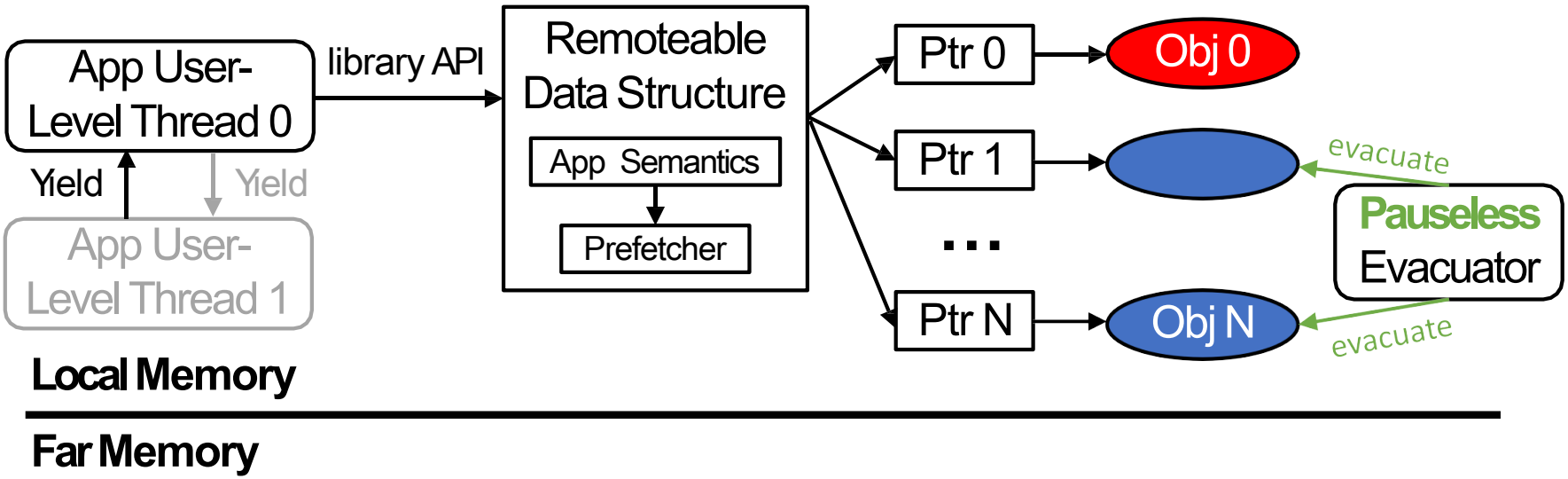
3. Pauseless Evacuator

➢ Solved challenge: impact of memory reclamation.



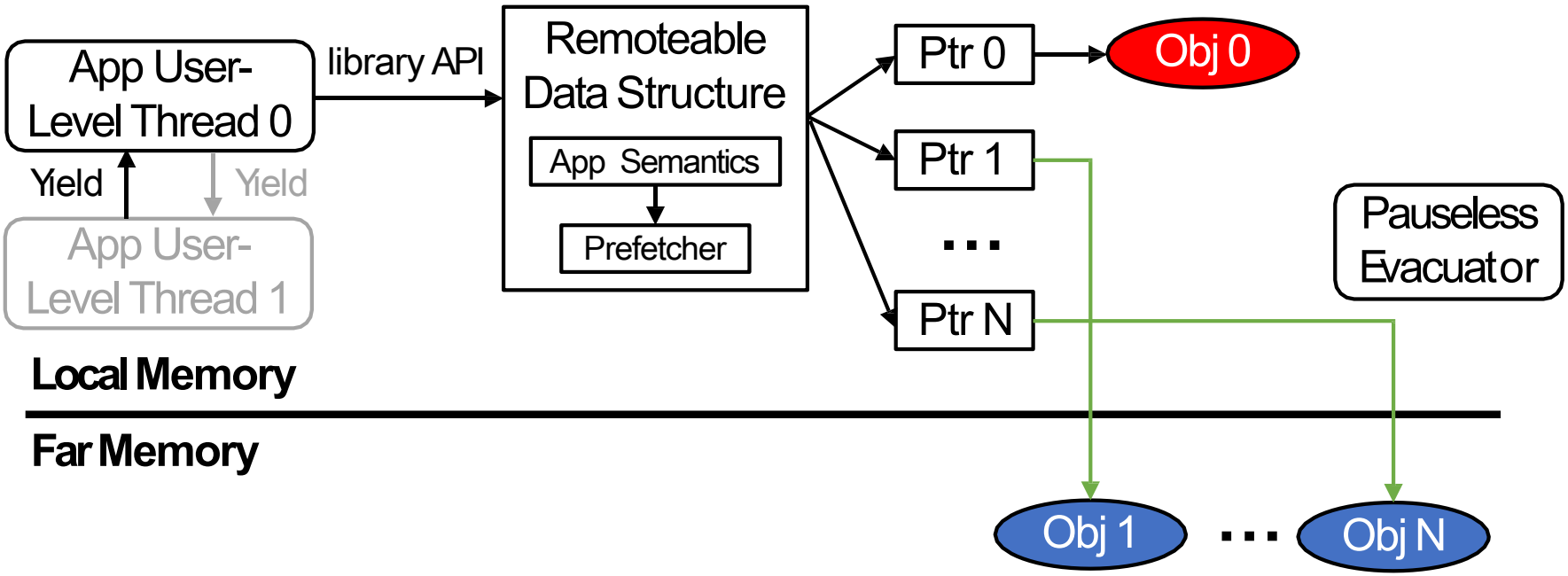
3. Pauseless Evacuator

➢Solved challenge: impact of memory reclamation.



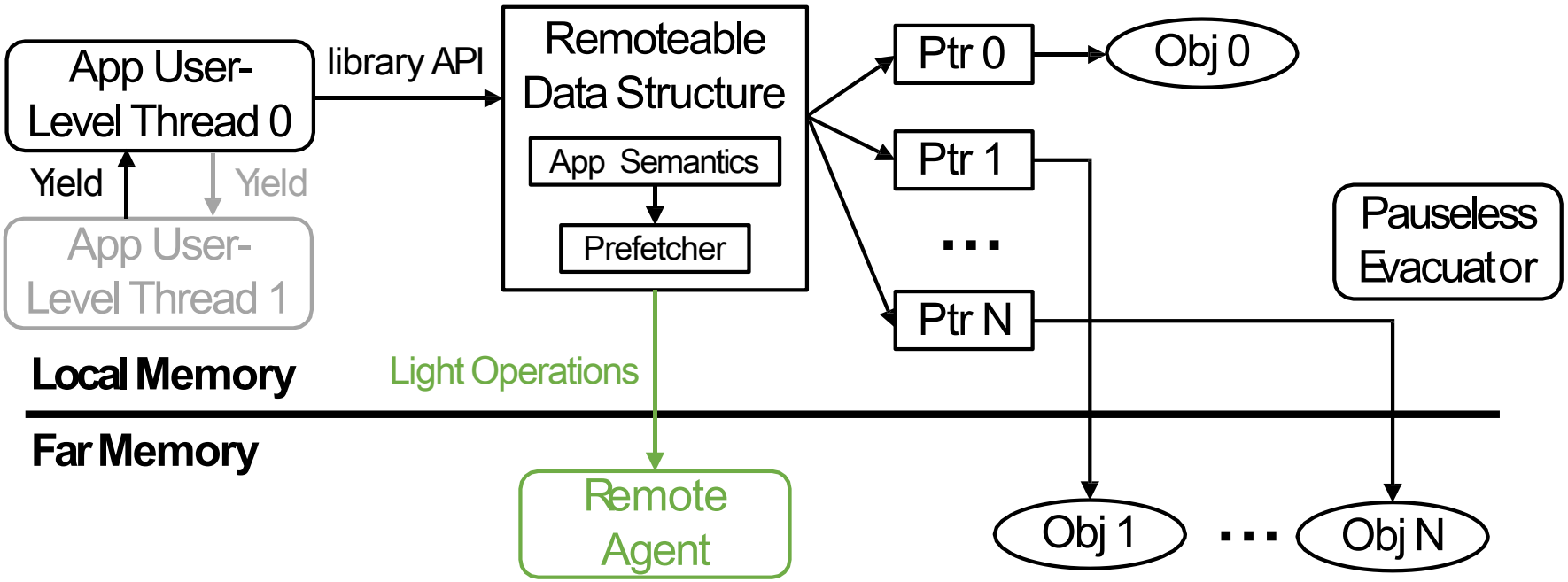
3. Pauseless Evacuator

➢Solved challenge: impact of memory reclamation.



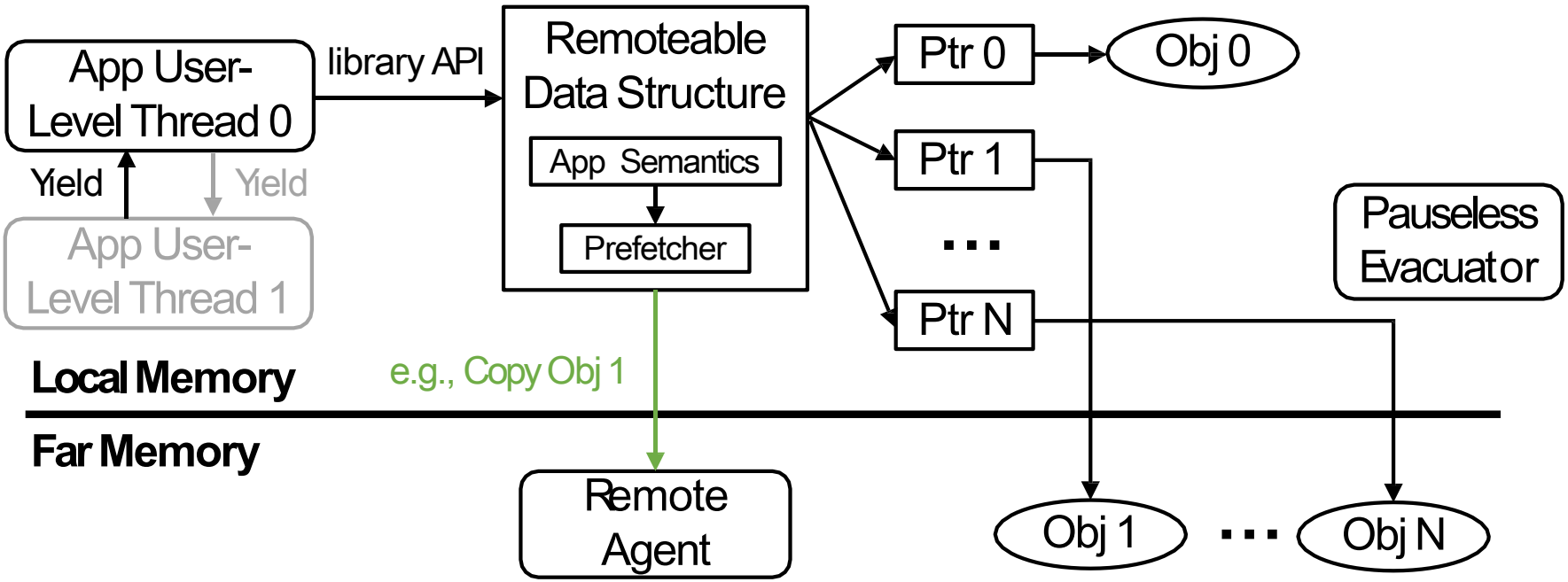
4. Remote Agent

➢ Solved challenge: network BW < DRAM BW.



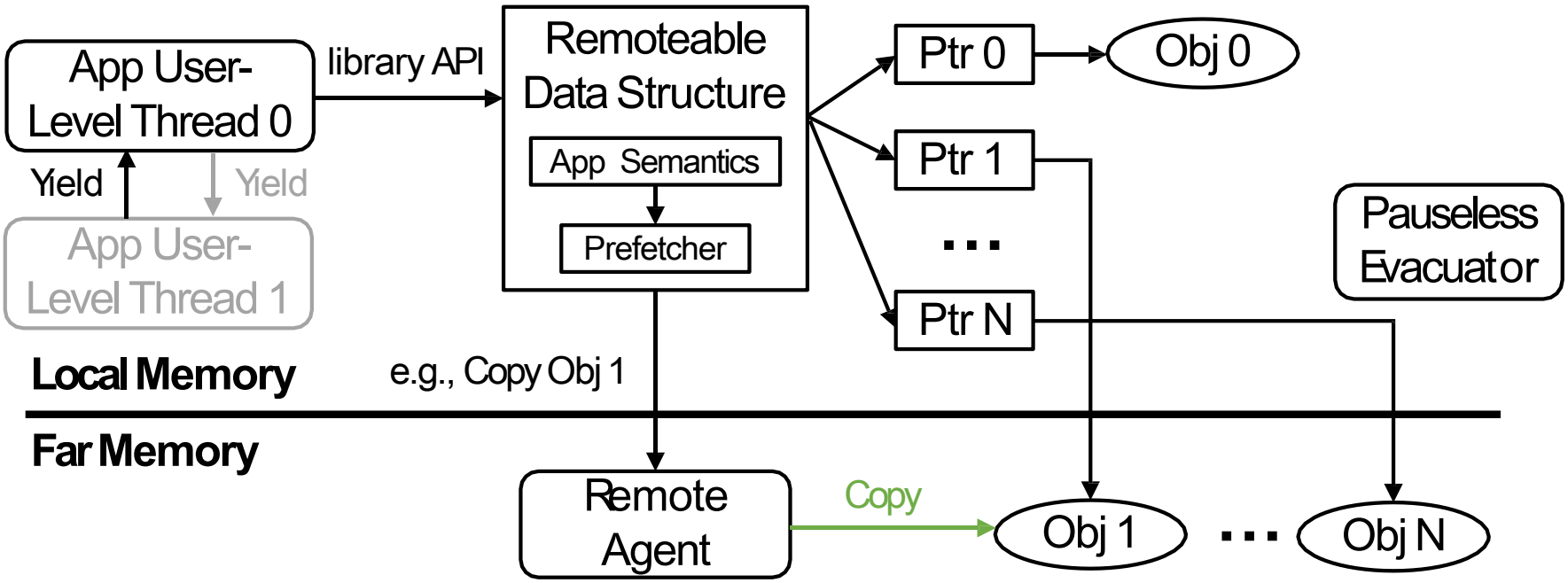
4. Remote Agent

➢ Solved challenge: network BW < DRAM BW.



4. Remote Agent

➢ Solved challenge: network BW < DRAM BW.



Sample Code

```
std::unordered_map<key_t, int> hashtable;
std::array<LargeData> arr;

LargeData foo(std::list<key_t> &keys_list) {
   int sum = 0;
   for (auto key : keys_list) {

      sum += hashtable.at(key);
   }

   LargeData ret = arr.at(sum);
   return ret;
}
```

Sample Code

```
RemHashTable<key_t, int> hashtable;
RemArray<LargeData> arr;

LargeData foo(RemList<key_t> &keys_list) {
   int sum = 0;
   for (auto key : keys_list) {

      sum += hashtable.at(key);
   }

   LargeData ret = arr.at(sum);
   return ret;
}
```

Sample Code

```
RemHashTable<key_t, int> hashtable;
RemArray<LargeData> arr;

LargeData foo(RemList<key_t> &keys_list) {
    int sum = 0;
    for (auto key : keys_list) {
        DerefScope scope;
        sum += hashtable.at(key, scope);
    }
    DerefScope scope;
    LargeData ret = arr.at(sum, scope);
    return ret;
}
```

Ensure the accessed objects will not be moved by the evacuator.

Sample Code

```
RemHashTable<key_t, int> hashtable;
RemArray<LargeData> arr;

LargeData foo(RemList<key_t> &keys_list) {
    int sum = 0;
    for (auto key : keys_list) {
        DerefScope scope;
        sum += hashtable.at(key, scope);
    }
    DerefScope scope;
    LargeData ret = arr.at</*don't cache*/ true>(sum, scope);
    return ret;
}
```

Sample Code

```
RemHashTable<key_t, int> hashtable;
RemArray<LargeData> arr;

LargeData foo(RemList<key_t> &keys_list) {
  int sum = 0;
  for (auto key : keys_list) {                    Prefetch list data.
    DerefScope scope;
    sum += hashtable.at(key, scope);
  }
  DerefScope scope;
  LargeData ret = arr.at</*don't cache*/ true>(sum, scope);
  return ret;
}
```

Sample Code

```
RemHashTable<key_t, int> hashtable;
RemArray<LargeData> arr;

LargeData foo(RemList<key_t> &keys_list) {
  int sum = 0;
  for (auto key : keys_list) {                    Prefetch list data.
    DerefScope scope;
    sum += hashtable.at(key, scope);              Cache hot objects.
  }
  DerefScope scope;
  LargeData ret = arr.at</*don't cache*/ true>(sum, scope);
  return ret;
}
```

Sample Code

```
RemHashTable<key_t, int> hashtable;
RemArray<LargeData> arr;

LargeData foo(RemList<key_t> &keys_list) {
  int sum = 0;
  for (auto key : keys_list) {                    Prefetch list data.
    DerefScope scope;
    sum += hashtable.at(key, scope);              Cache hot objects.
  }
  DerefScope scope;
  LargeData ret = arr.at</*don't cache*/ true>(sum, scope);   Avoid polluting local mem.
  return ret;
}
```

Implementation

➢ Implemented 6 data structures.
- Array, List, Hashtable, Vector, Stack, and Queue.

Implementation

- Implemented 6 data structures.
  - Array, List, Hashtable, Vector, Stack, and Queue.

➢ Runtime is built on top of Shenango [NSDI'19].

Implementation

- Implemented 6 data structures.
  - Array, List, Hashtable, Vector, Stack, and Queue.
- Runtime is built on top of Shenango [NSDI'19].

➢TCP far-memory backend.

Implementation

- Implemented 6 data structures.
  - Array, List, Hashtable, Vector, Stack, and Queue.
- Runtime is built on top of Shenango [NSDI'19].
- TCP far-memory backend.

➢ LoC: 6.5K (runtime) + 5.5K (data structures) + 0.8K (Shenango)

Evaluation

➢Setup: 1 compute server + 1 far memory server, 25 GbE.

Evaluation

- Setup: 1 compute server + 1 far memory server, 25 GbE.
- How does AIFM
  - ➢…perform on applications with different compute intensities?

Evaluation

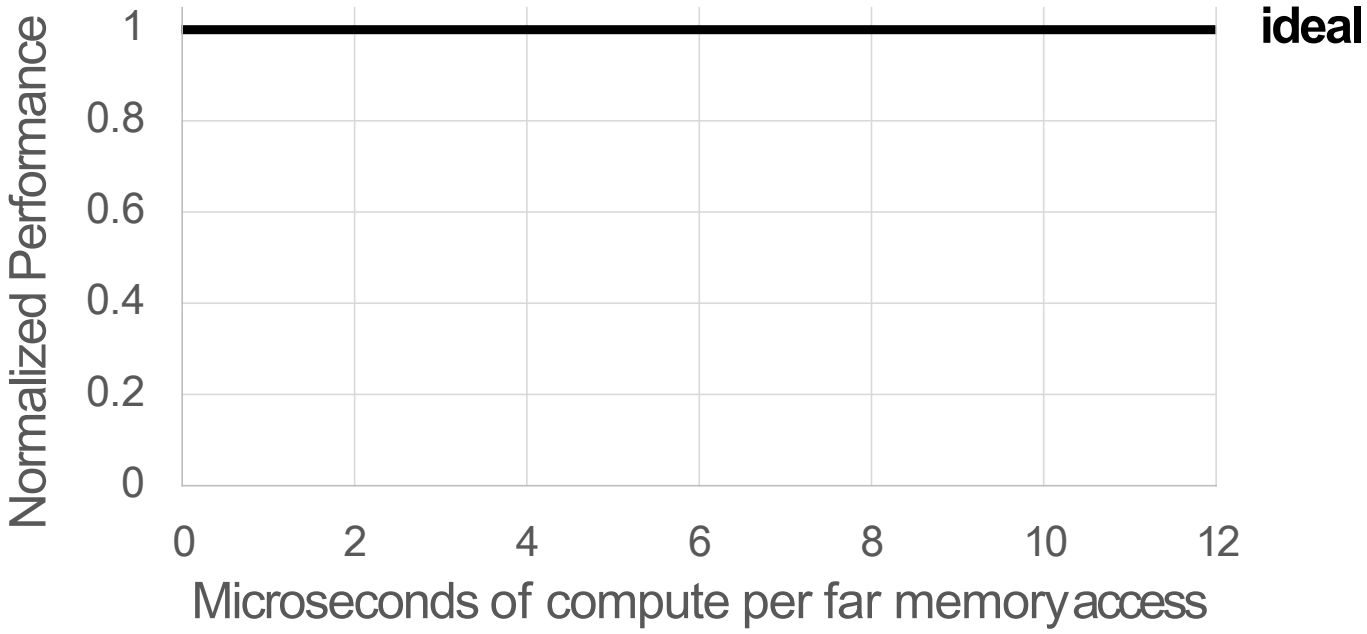
- Setup: 1 compute server + 1 far memory server, 25 GbE.
- How does AIFM
  - …perform on applications with different compute intensities?
  - ➢…compare to the local-only (ideal) system?

Evaluation

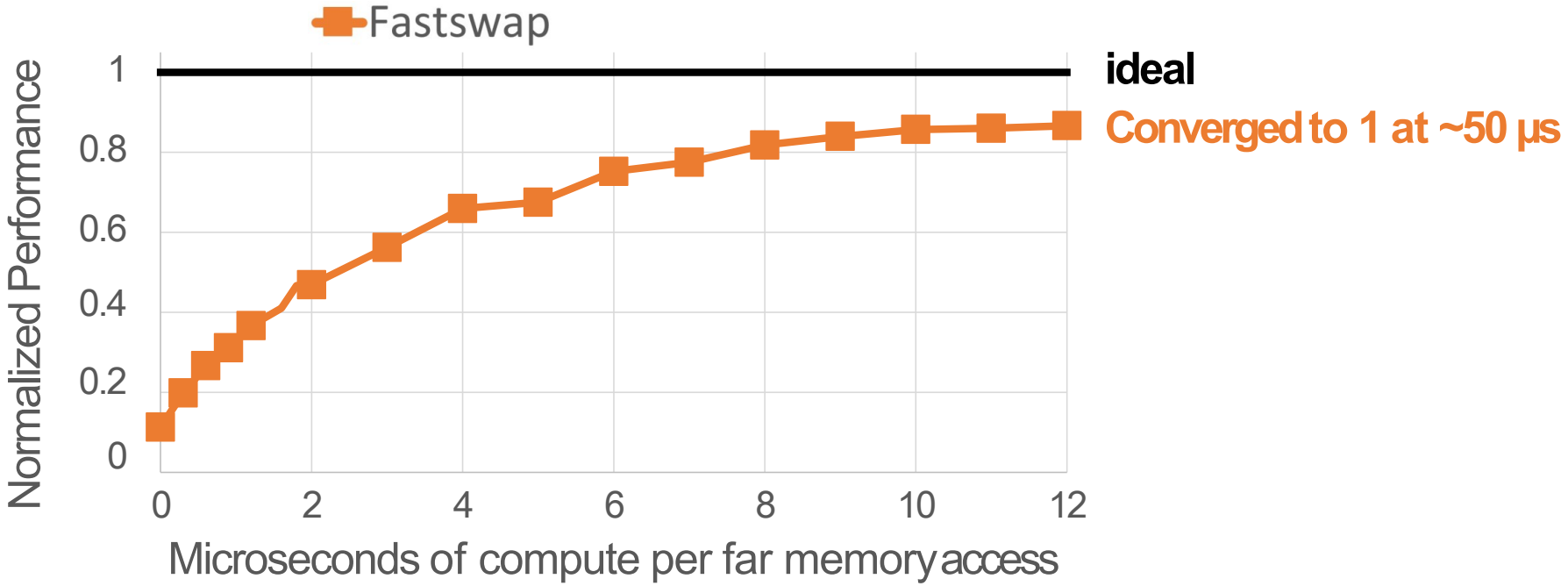
- Setup: 1 compute server + 1 far memory server, 25 GbE.
- How does AIFM
  - …perform on applications with different compute intensities?
  - …compare to the local-only (ideal) system?

  ➢ …compare to the state-of-the-art paging system, Fastswap [EuroSys'20]?

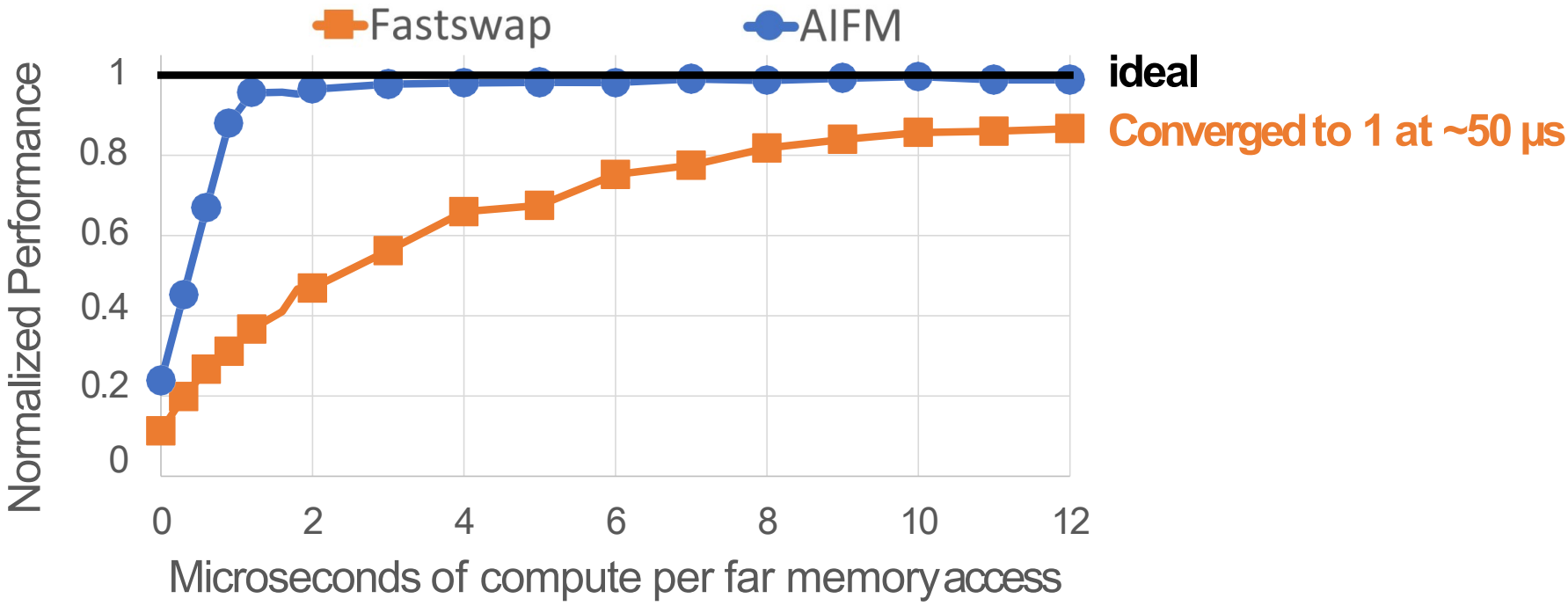
Performance on Different Compute Intensities



Performance on Different Compute Intensities

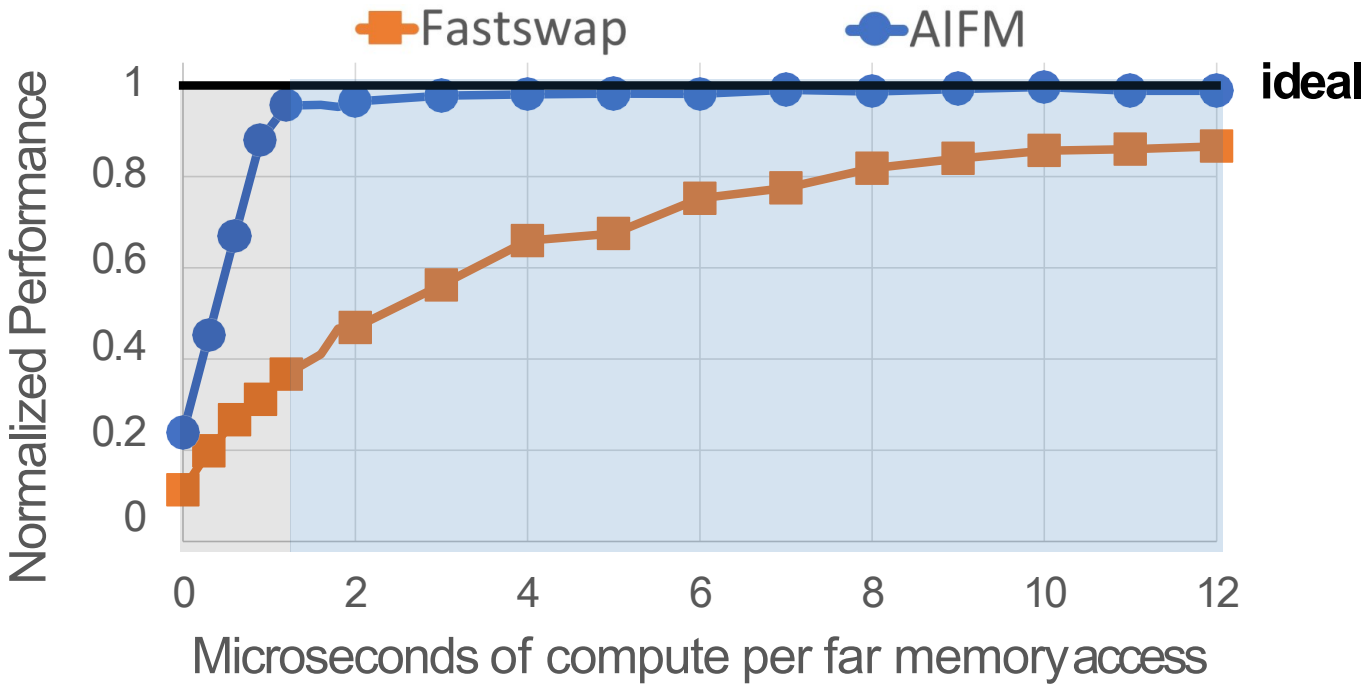


Performance on Different Compute Intensities



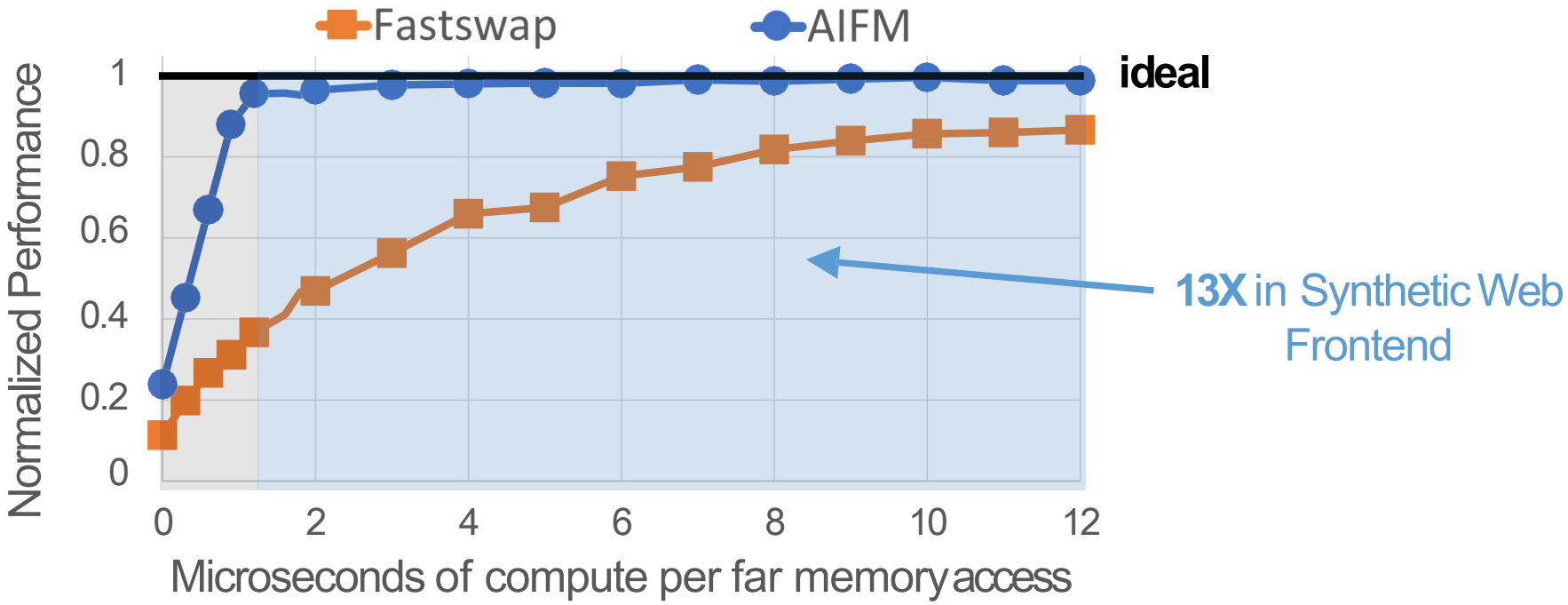
AIFM hides far memory latency with moderate compute.

Performance on Different Compute Intensities

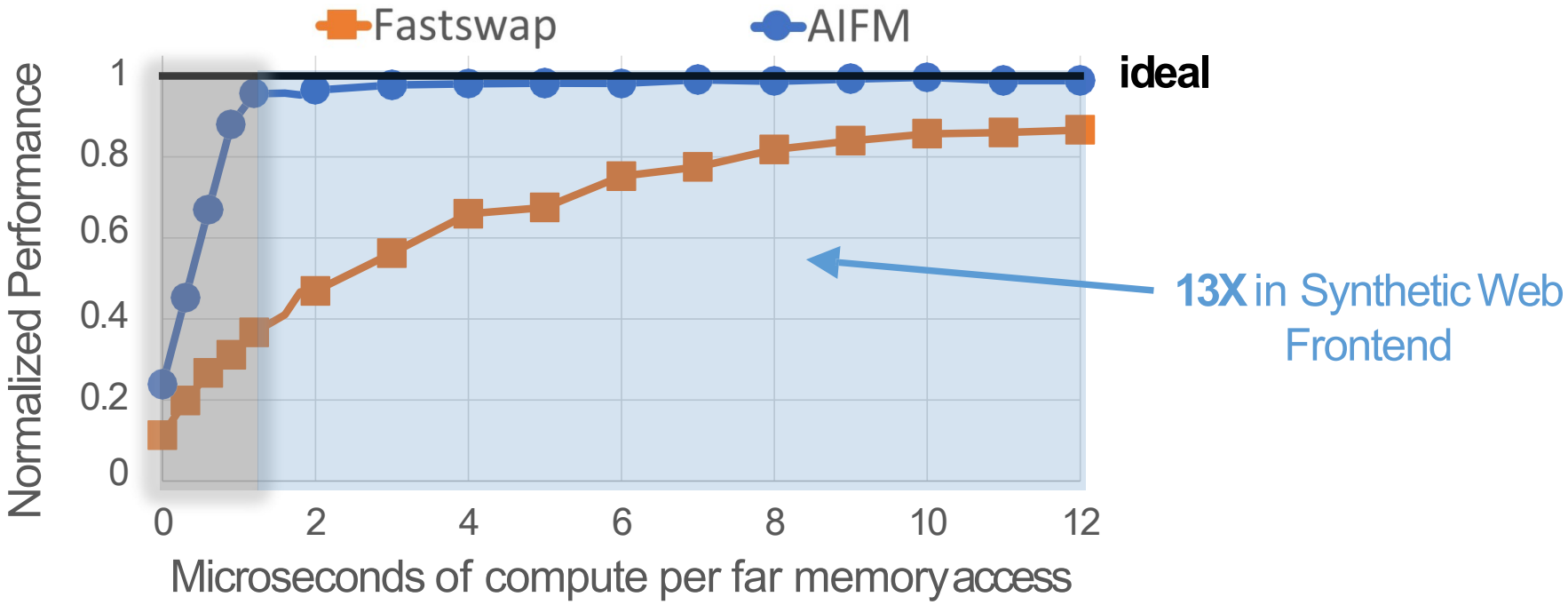


AIFM hides far memory latency with moderate compute.

Performance on Different Compute Intensities



Performance on Different Compute Intensities



Conclusion

- AIFM: Application-Integrated Far Memory.
- Key idea: swap memory using a userspace runtime.
  - Data Structure Library: captures application semantics.
  - Userspace Runtime: efficiently manages objects and memory.
- Achieves 13X end-to-end speedup over Fastswap.

➢ Code released at https://github.com/AIFM-sys/AIFM

Please send your questions to us

zainruan@csail.mit.edu

Table of contents

- Expose semantics to manage far memory better
  - **AIFM**: High-Performance Applications-Integrated Far Memory, **OSDI'20**
- Reinvent paging for new workloads:
  - Efficient Memory Management for Large Language Model Serving with **PagedAttention**, **SOSP'23**
- Predict accesses before demand misses happen
  - **PF-LLM**: Large Language Model Hinted Hardware Prefetching, **ASPLOS'26 (Best Paper)**

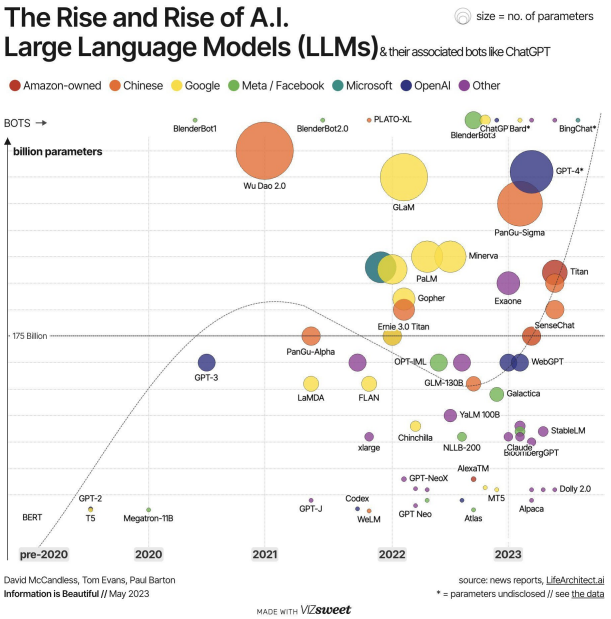
Efficient Memory Management for Large Language Model Serving with *PagedAttention*

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. Proceedings of the 29th Symposium on Operating Systems Principles, 611–626. https://doi.org/10.1145/3600006.3613165

Serving LLMs is Expensive

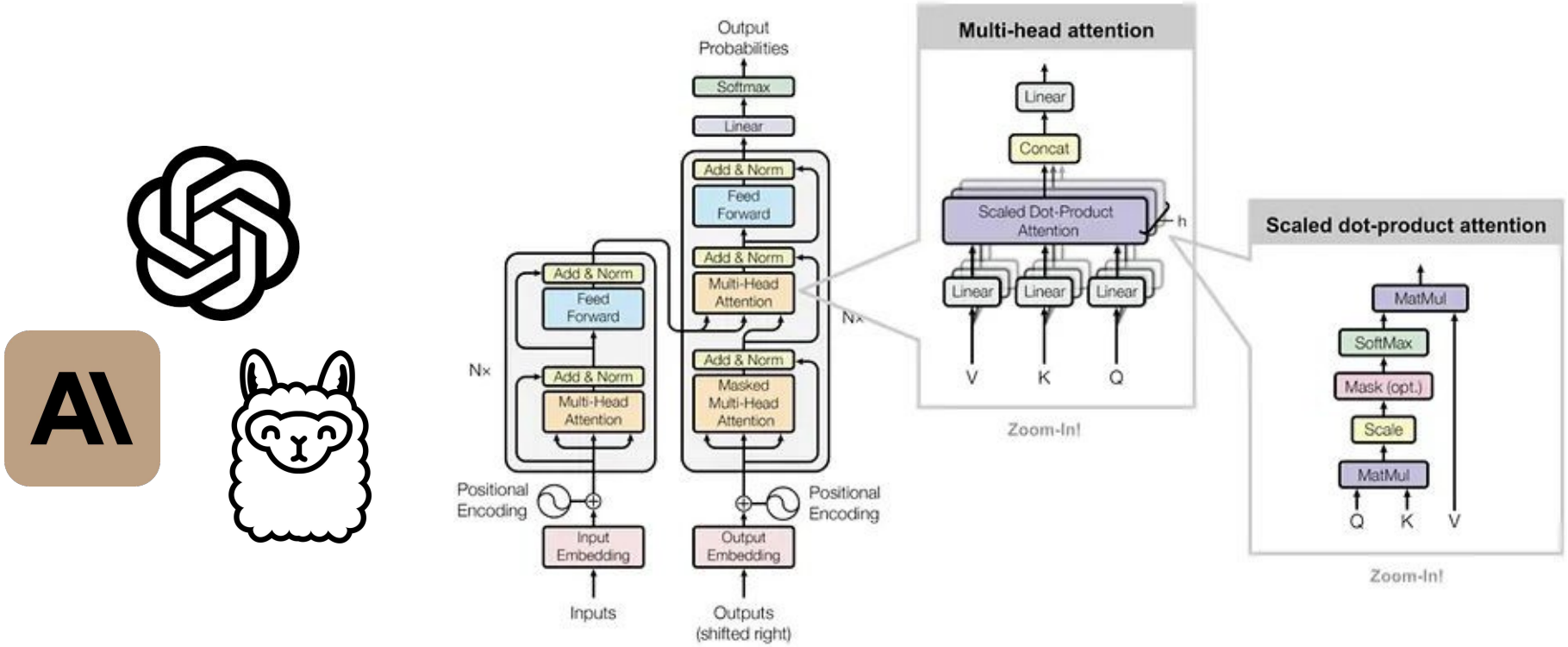
- A ton of GPUs are required for production scale LLM services
- Nevertheless, each GPU only serve a handful of requests per second
  - For LLaMA-13B and moderate-size inputs, 1 A100 can process < 1 requests per second



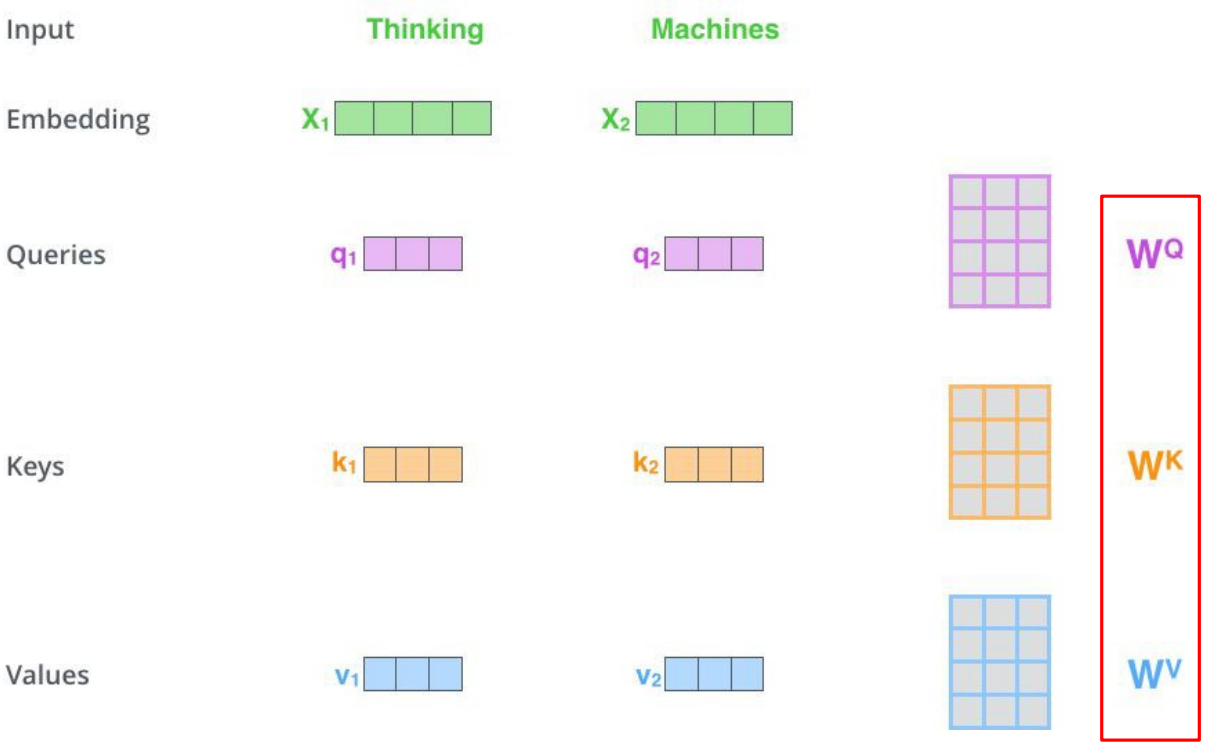
Zuckerberg's Meta Is Spending Billions to Buy 350,000 Nvidia H100 GPUs

In total, Meta will have the compute power equivalent to 600,000 Nvidia H100 GPUs to help it develop next-generation AI, says CEO Mark Zuckerberg.

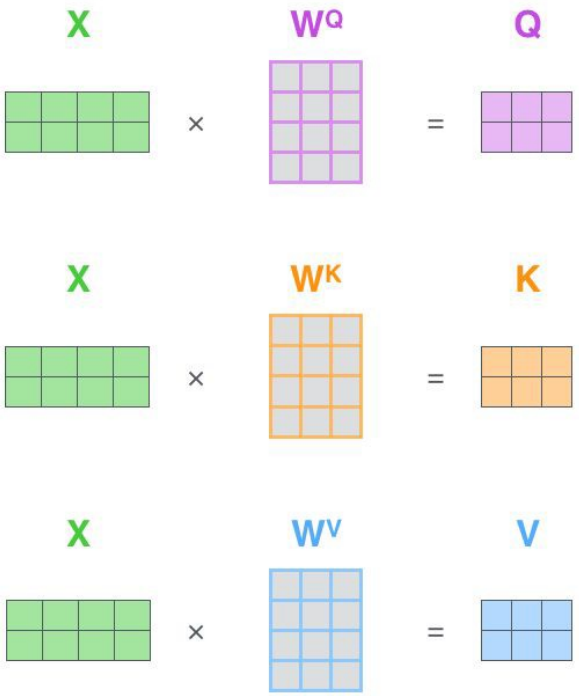
Related work: KV Cache



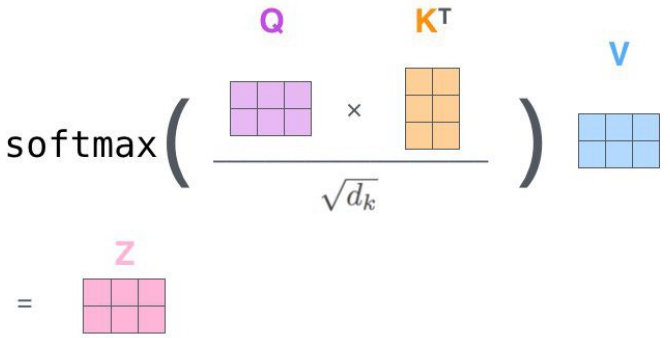
Self-Attention



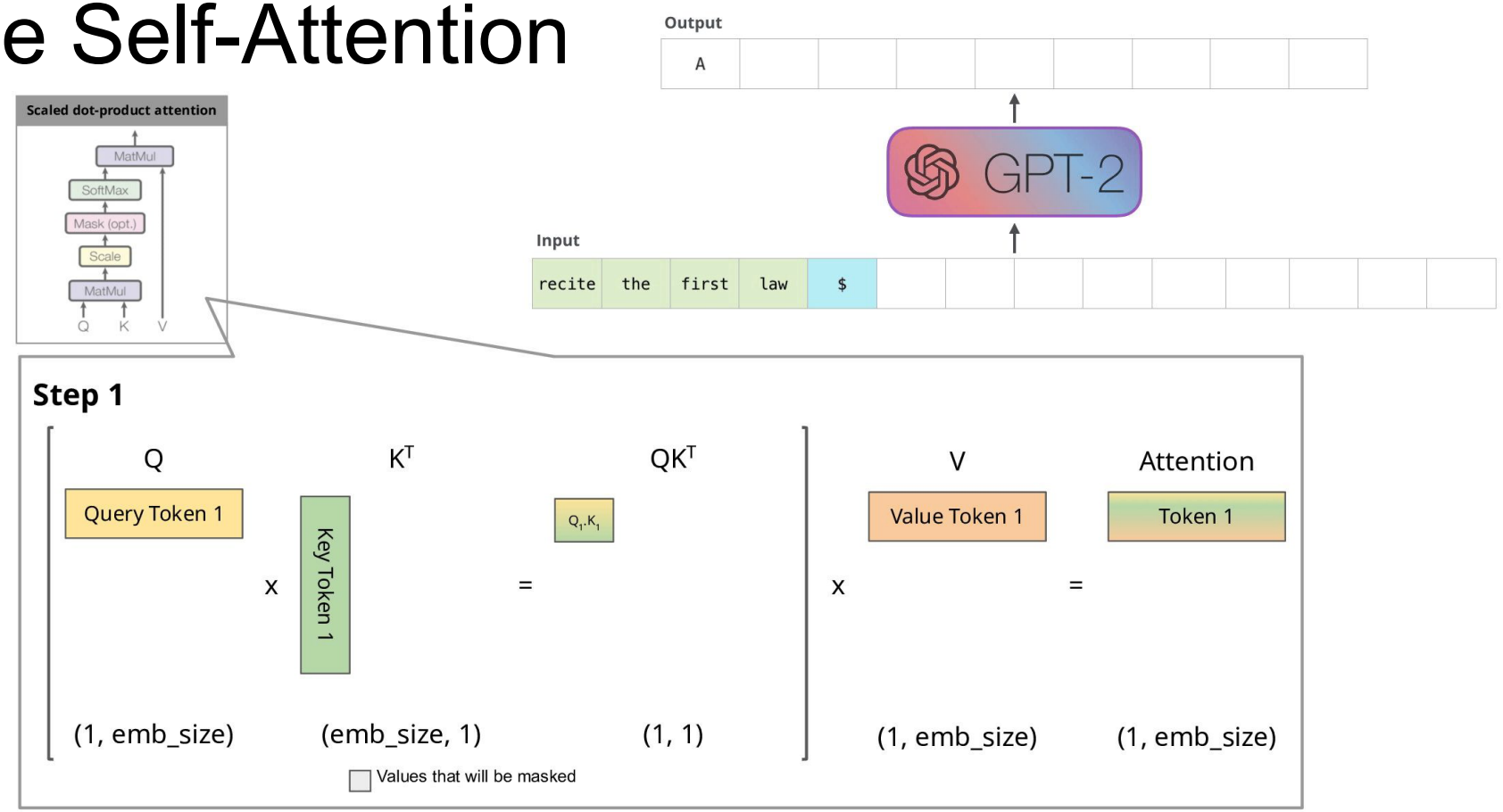
Self-Attention



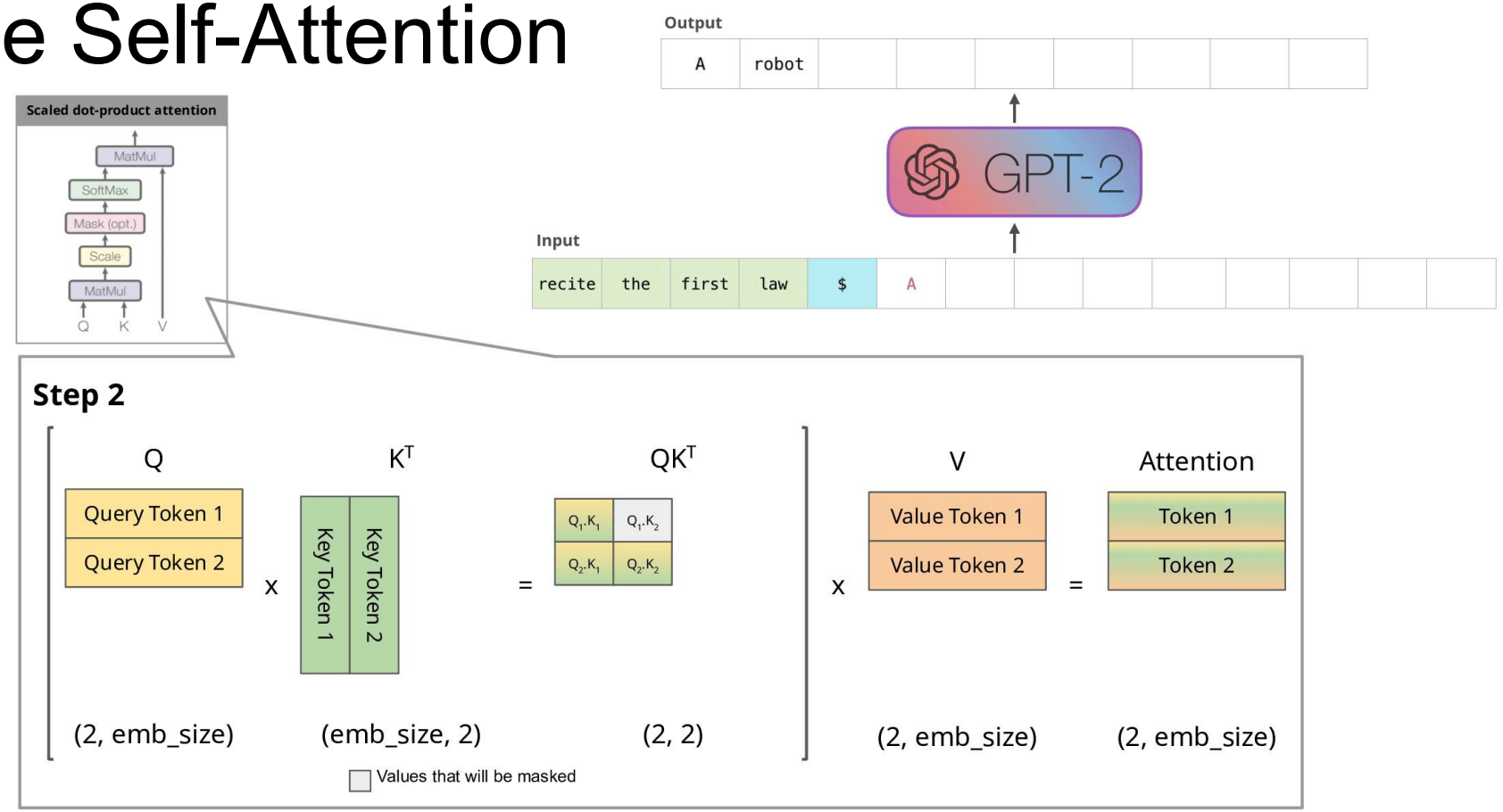
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$



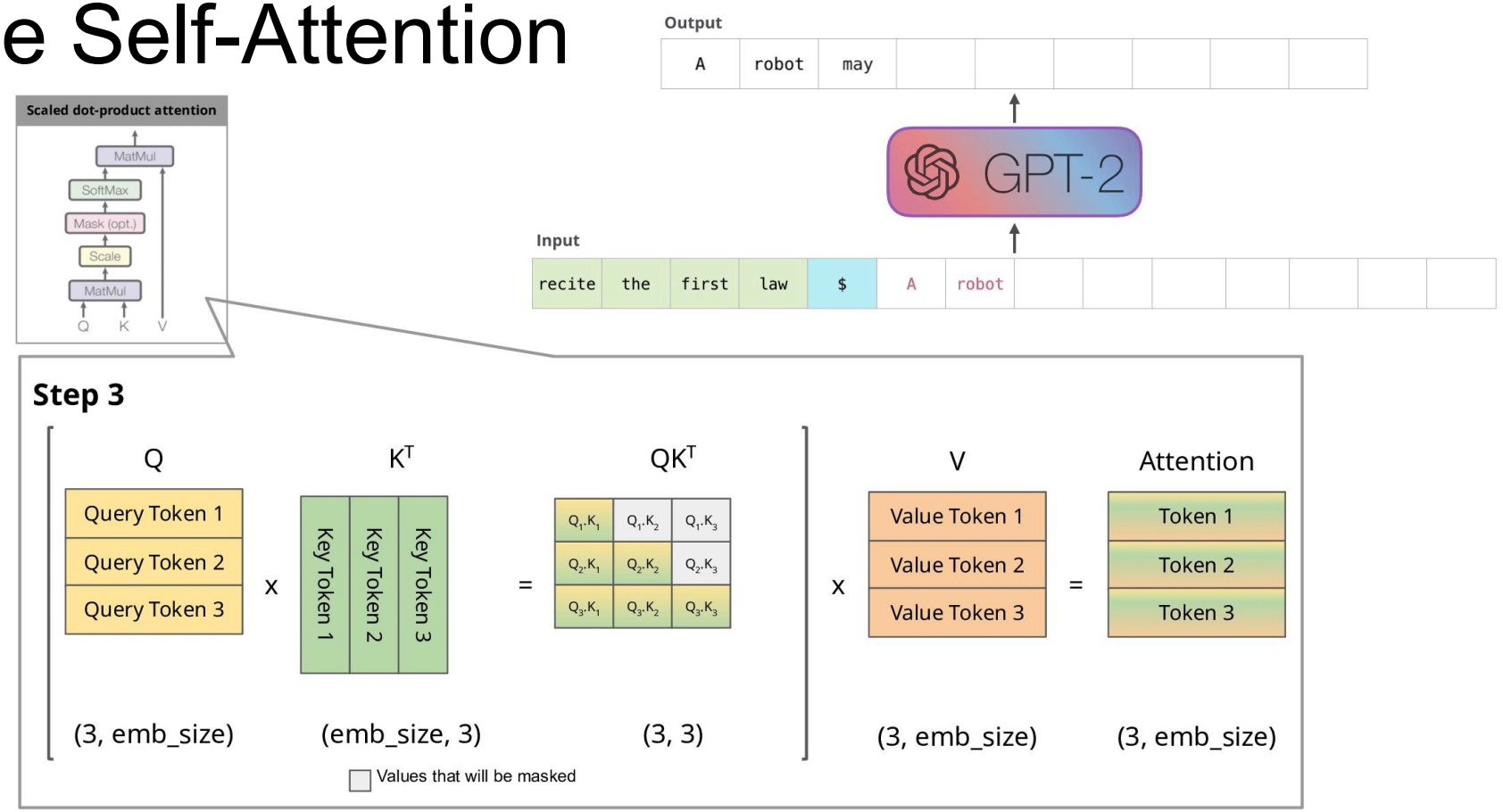
Naive Self-Attention



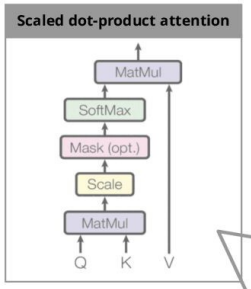
Naive Self-Attention



Naive Self-Attention



Naive Self-Attention



Output

A	robot	may	not					

GPT-2

Input

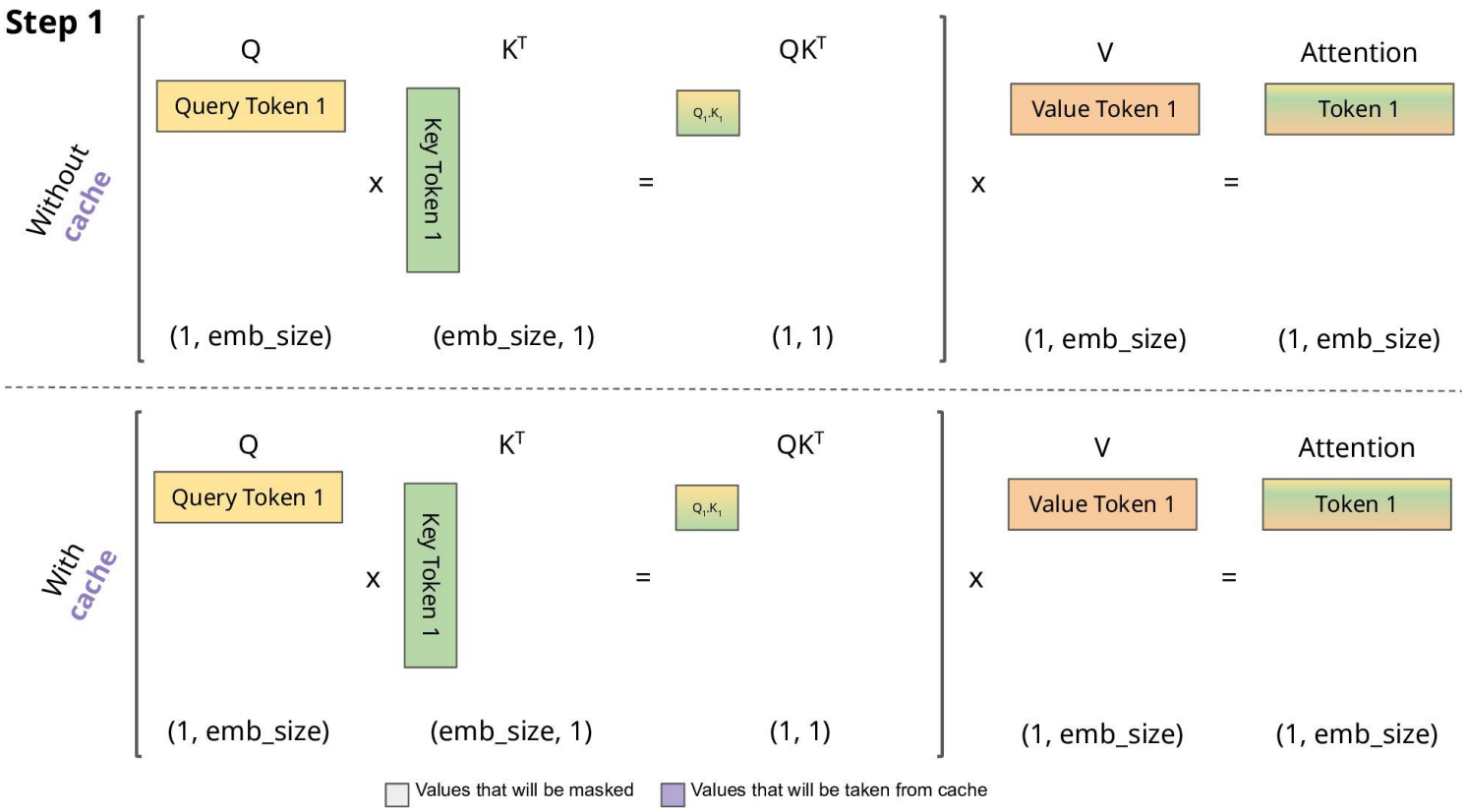
recite	the	first	law	$	A	robot	may						

Step 4

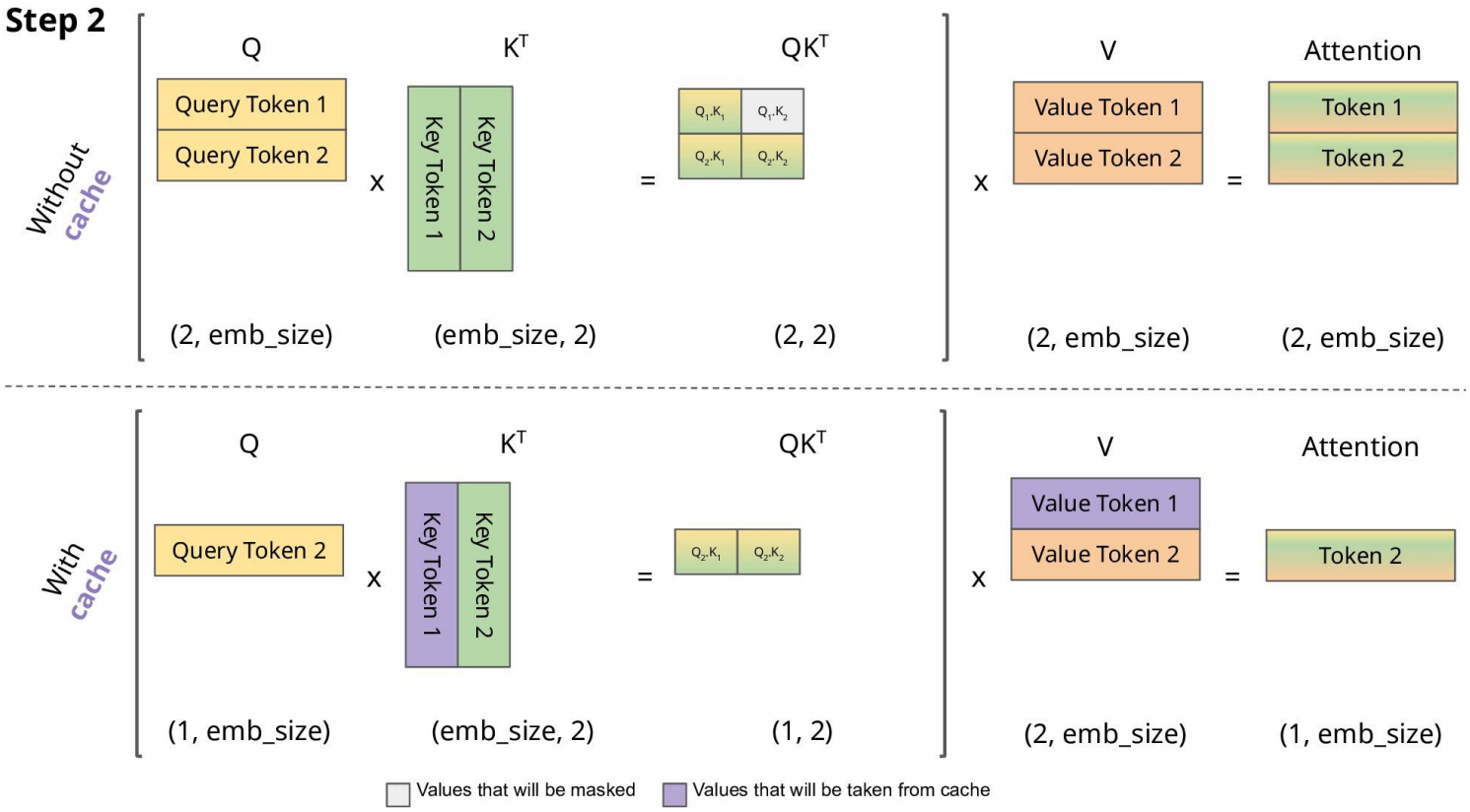
Q		K^T		QK^T		V		Attention
Query Token 1	x	Key Token 1 Key Token 2 Key Token 3 Key Token 4	=	$Q_1.K_1$ $Q_1.K_2$ $Q_1.K_3$ $Q_1.K_4$ $Q_2.K_1$ $Q_2.K_2$ $Q_2.K_3$ $Q_2.K_4$ $Q_3.K_1$ $Q_3.K_2$ $Q_3.K_3$ $Q_3.K_4$ $Q_4.K_1$ $Q_4.K_2$ $Q_4.K_3$ $Q_4.K_4$	x	Value Token 1 Value Token 2 Value Token 3 Value Token 4	=	Token 1 Token 2 Token 3 Token 4
Query Token 2								
Query Token 3								
Query Token 4								
(4, emb_size)		(emb_size, 4)		(4, 4)		(4, emb_size)		(4, emb_size)

Values that will be masked

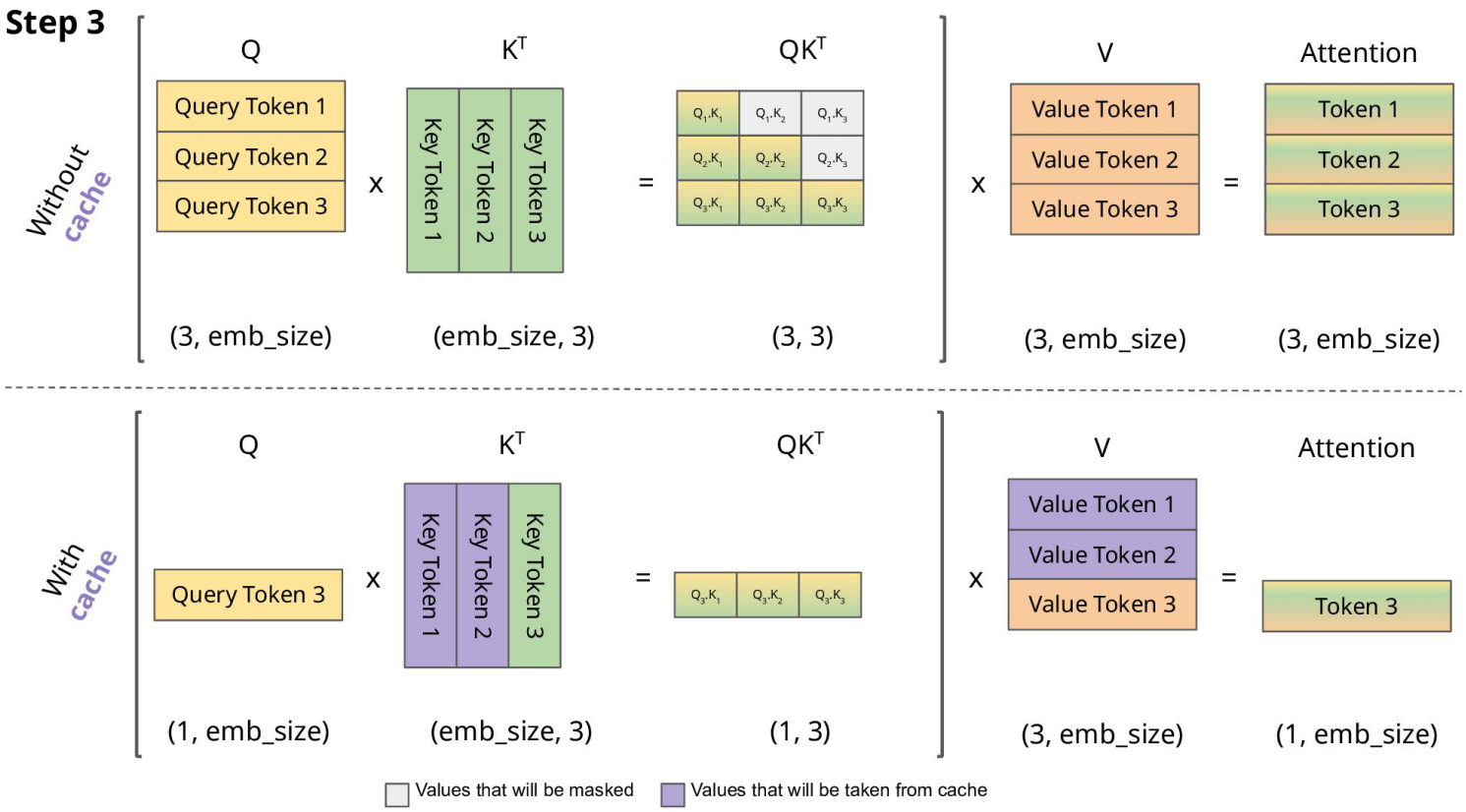
Self-Attention with KV Cache



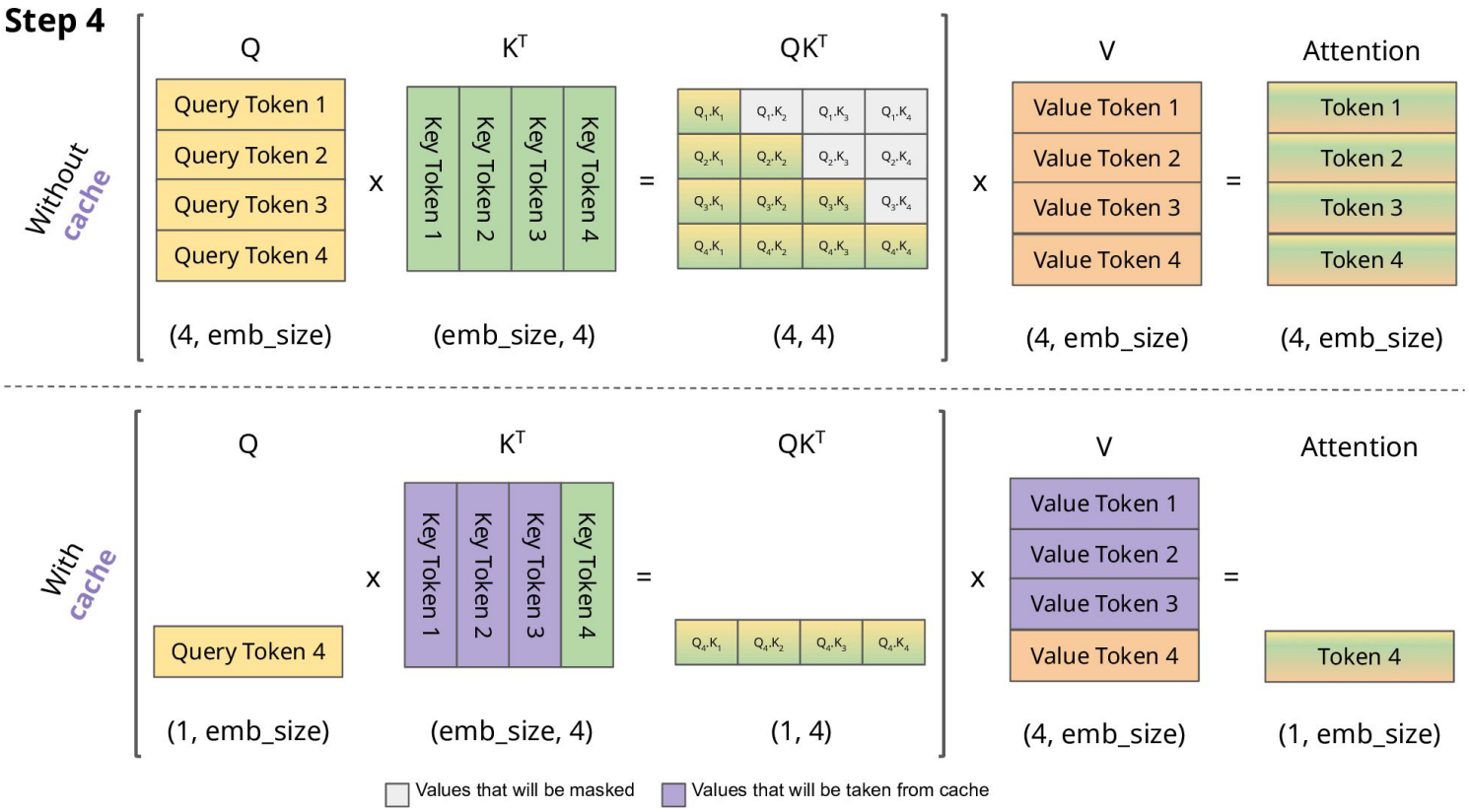
Self-Attention with KV Cache



Self-Attention with KV Cache



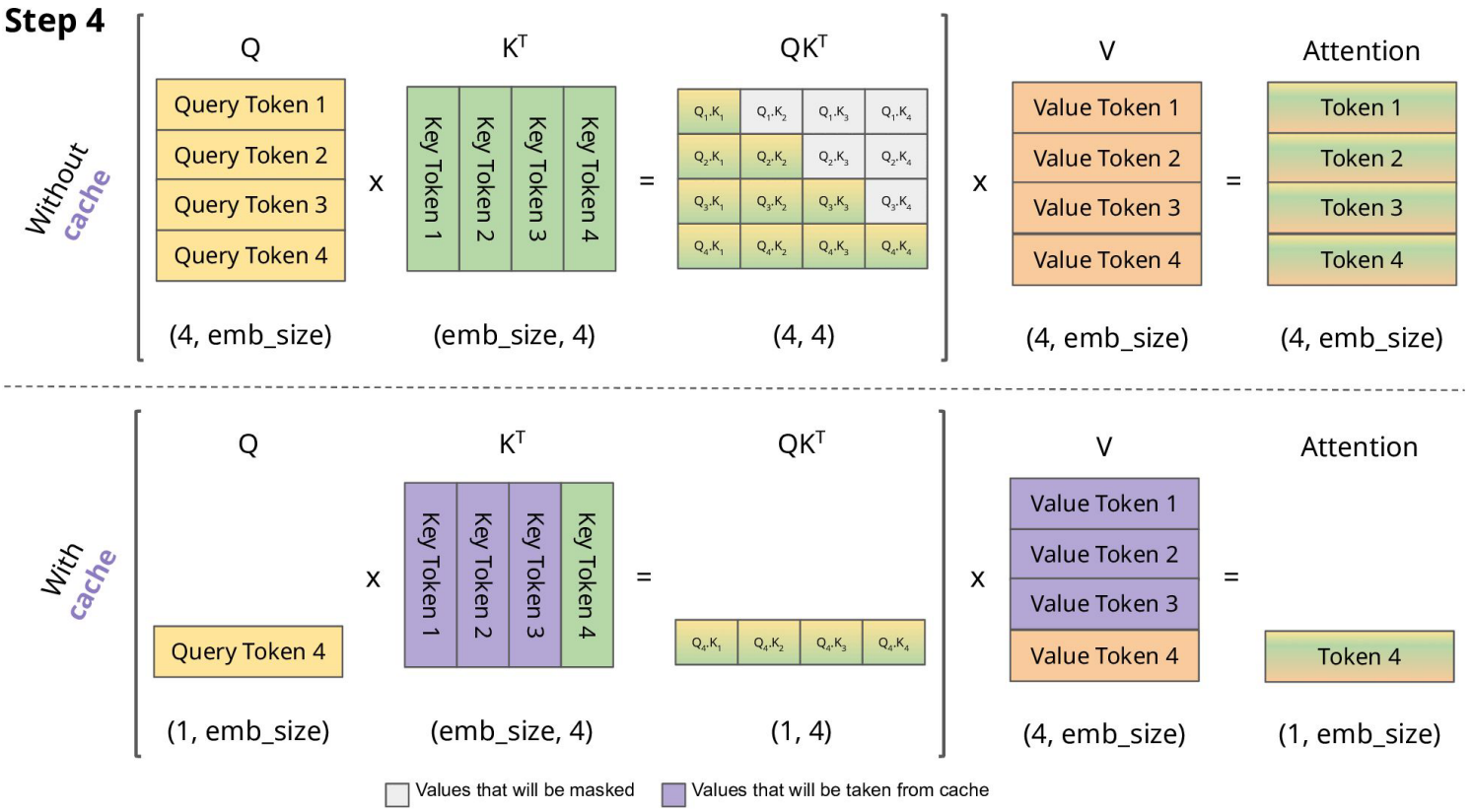
Self-Attention with KV Cache



Self-Attention with KV Cache

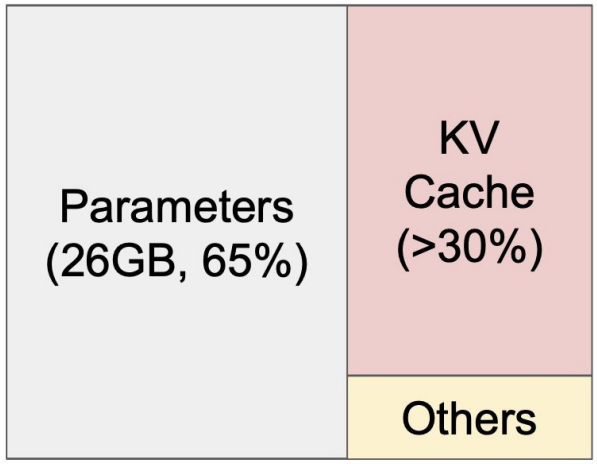
T : Sequence length

d : Model embedding size



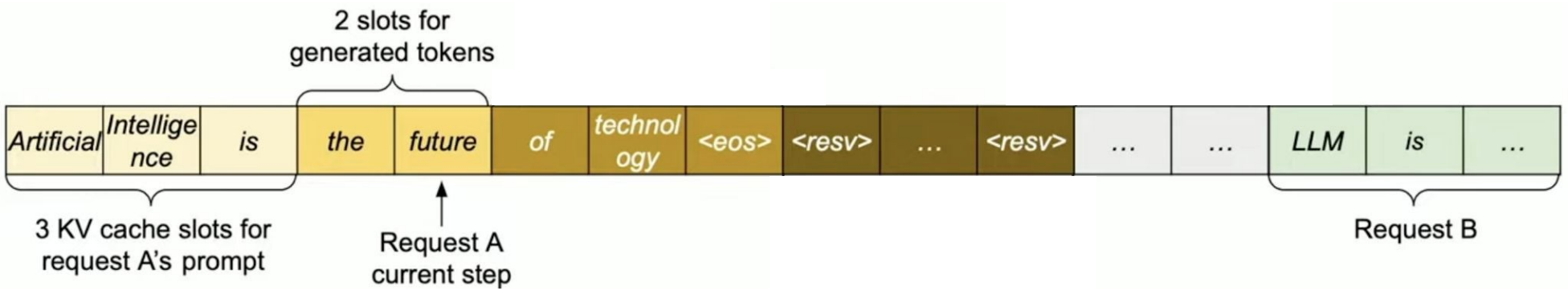
Memory layout when serving an LLM

- Efficient management of KV cache is crucial for high-throughput LLM serving

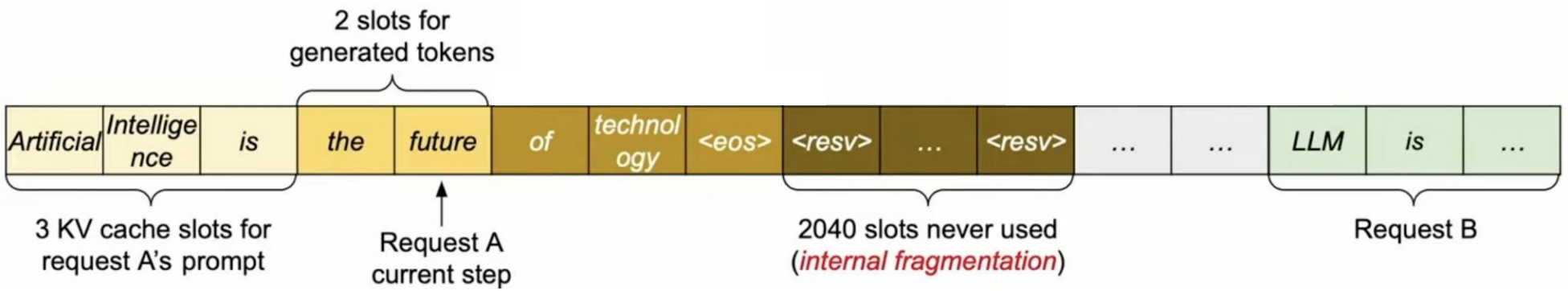


NVIDIA A100 40GB

Motivation: Memory waste in KV Cache

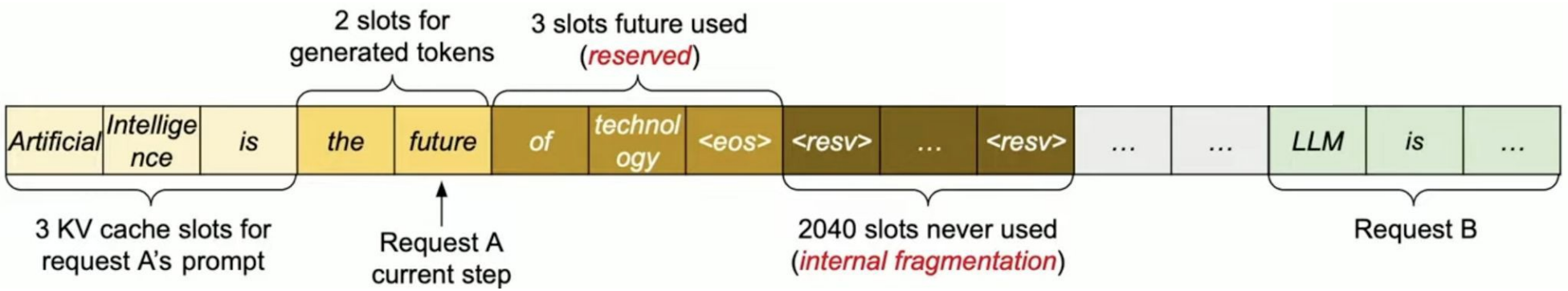


Motivation: Memory waste in KV Cache



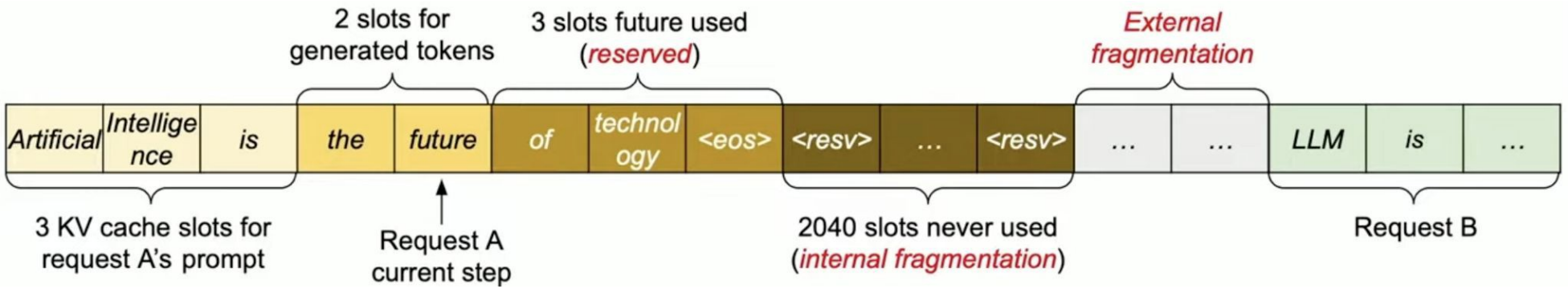
- **Internal fragmentation:** memory reserved for a request but left unused because the final output length is shorter than expected

Motivation: Memory waste in KV Cache



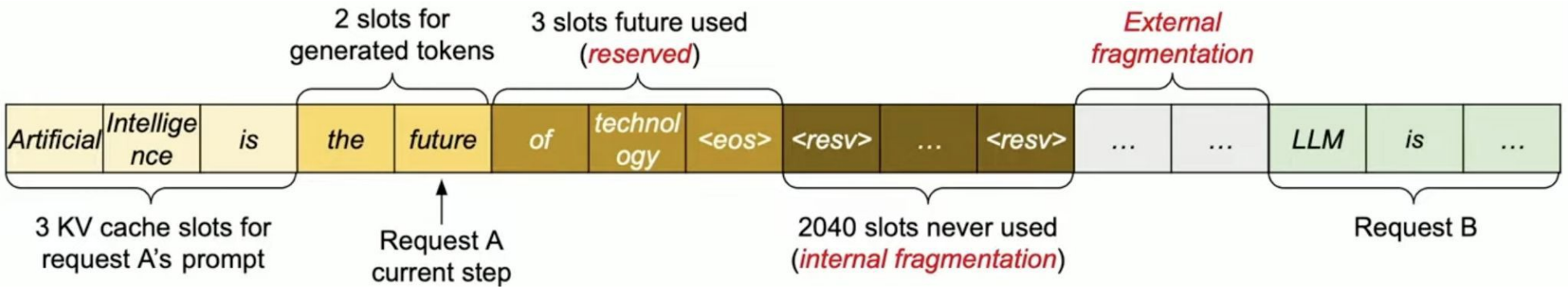
- **Internal fragmentation:** memory reserved for a request but left unused because the final output length is shorter than expected
- **Reservation:** memory currently unused but reserved for future use by the same request

Motivation: Memory waste in KV Cache



- **Internal fragmentation:** memory reserved for a request but left unused because the final output length is shorter than expected
- **Reservation:** memory currently unused but reserved for future use by the same request
- **External fragmentation:** small unused memory gaps between allocations caused by varying sequence lengths across different requests

Motivation: Memory waste in KV Cache

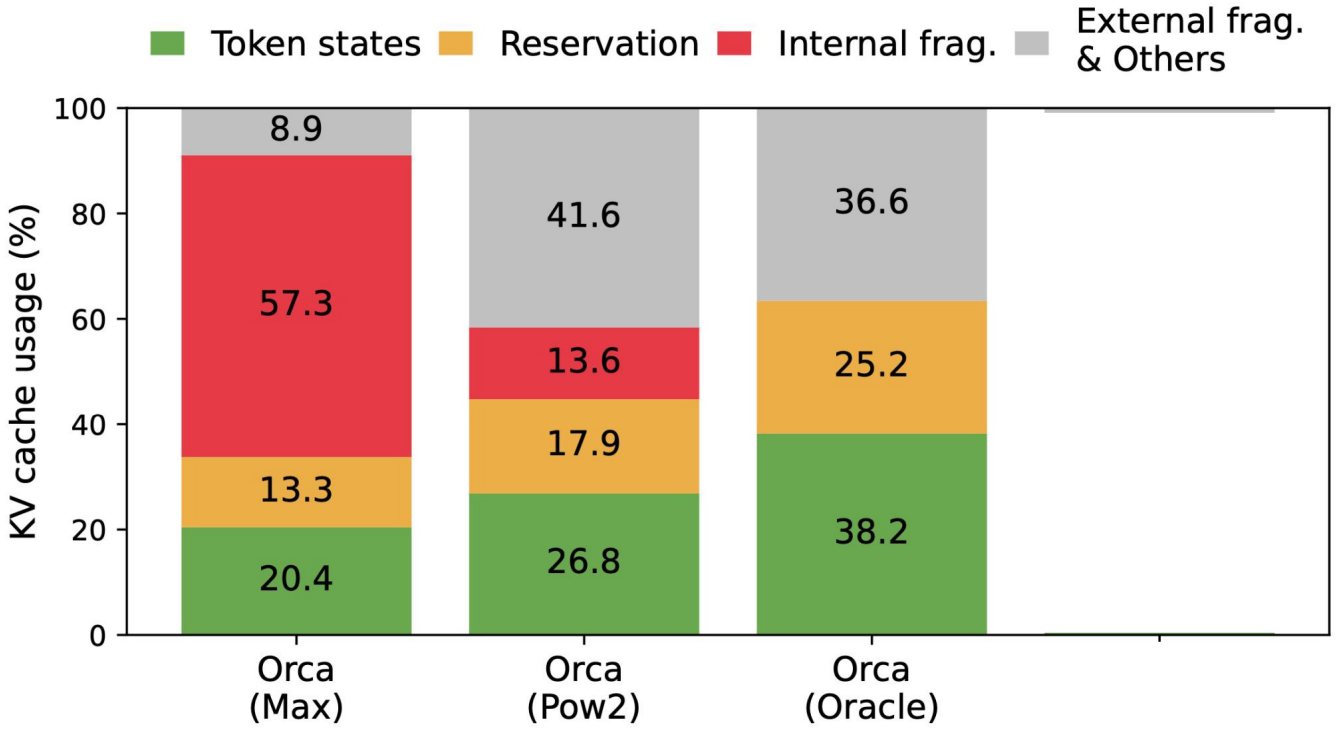


- **Internal fragmentation:** memory reserved for a request but left unused because the final output length is shorter than expected
- **Reservation:** memory currently unused but reserved for future use by the same request
- **External fragmentation:** small unused memory gaps between allocations caused by varying sequence lengths across different requests

Essentially **segmentation**-style memory management.

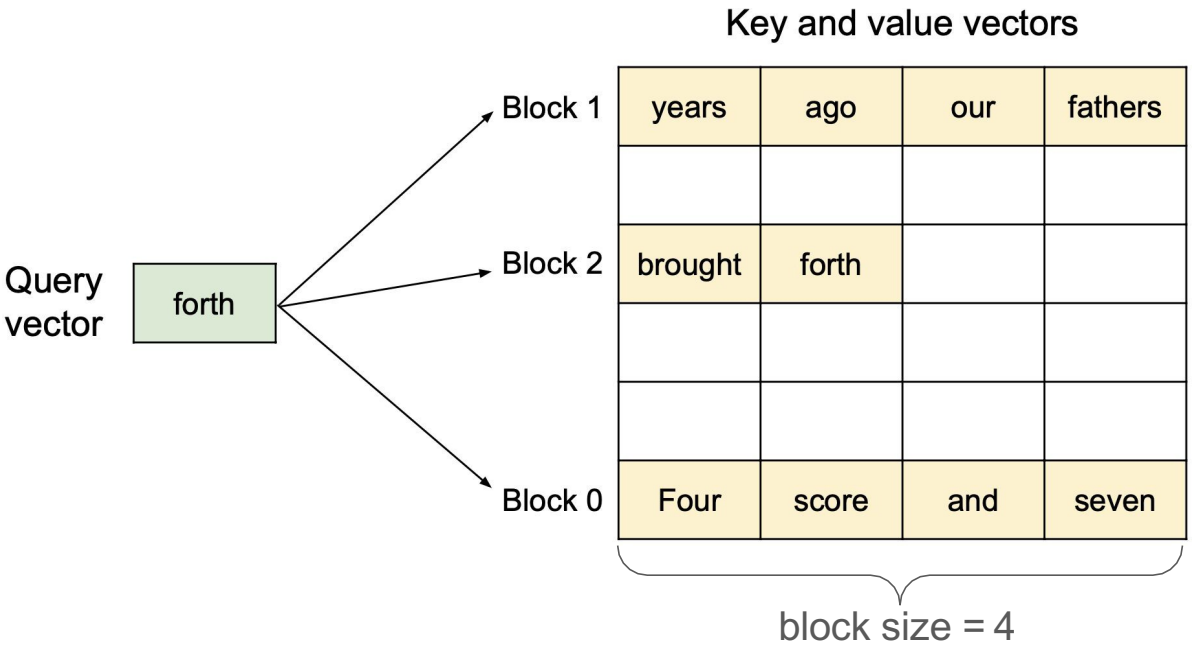
Motivation: Memory waste in KV Cache

- Only 20-40% of KV cache is used to store the actual token states



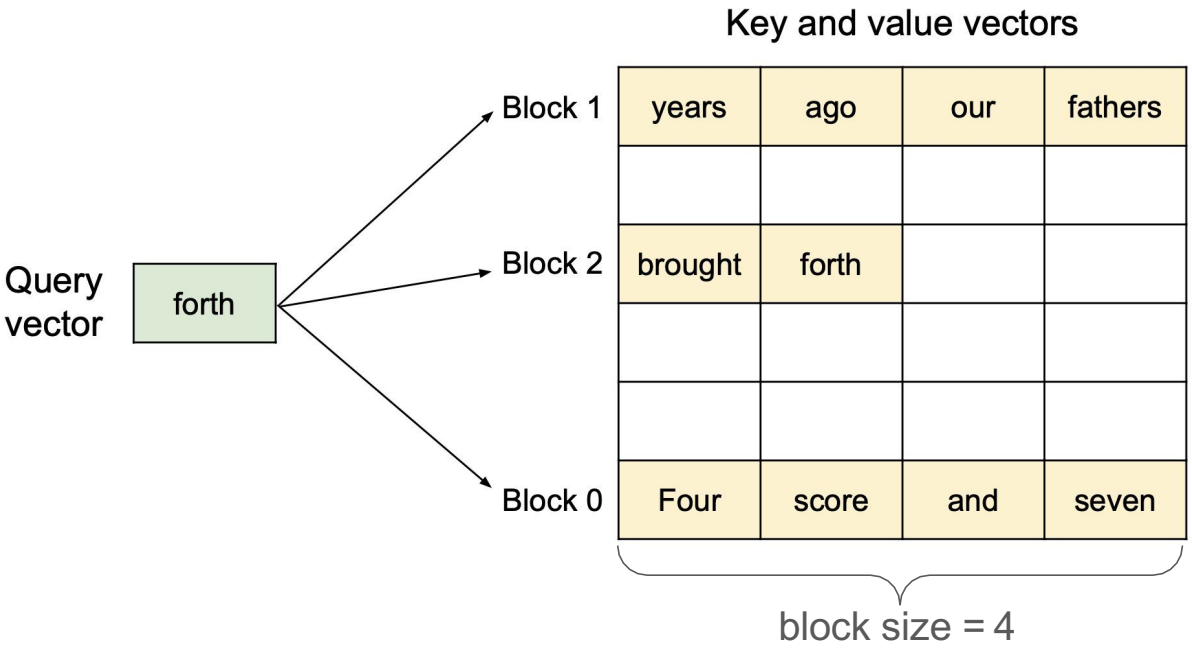
vLLM: Efficient memory management using PagedAttention

- PagedAttention algorithm allows for storing attention key and values vectors in non-contiguous blocks in the memory
- Each KV block is a fixed-size contiguous chunk of memory that can store KV cache from left to right
- Attention for each new token is computed across all tokens in the non-contiguous KV blocks

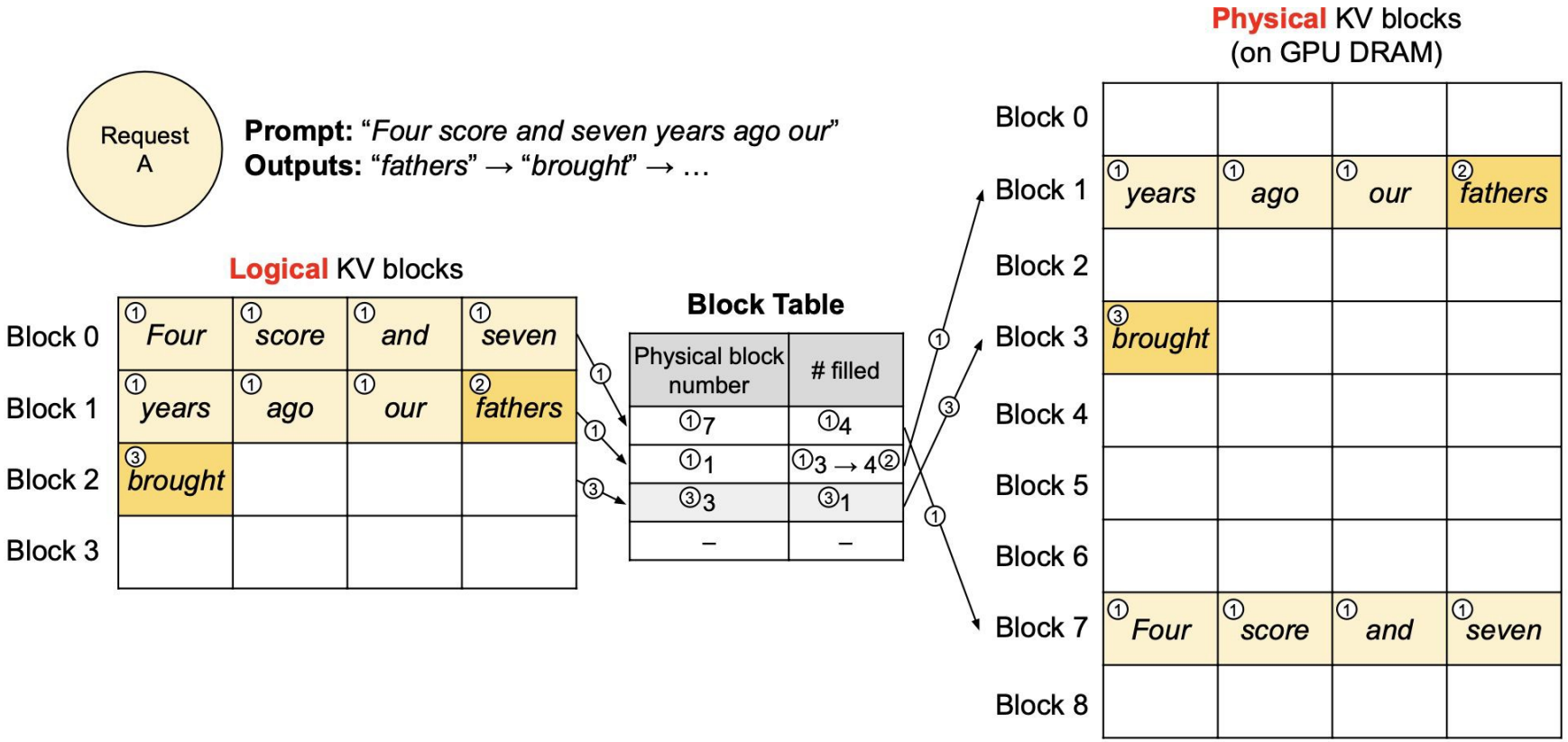


vLLM: Efficient memory management using PagedAttention

Now let's re-invent **paging** in the context of KV cache ☺

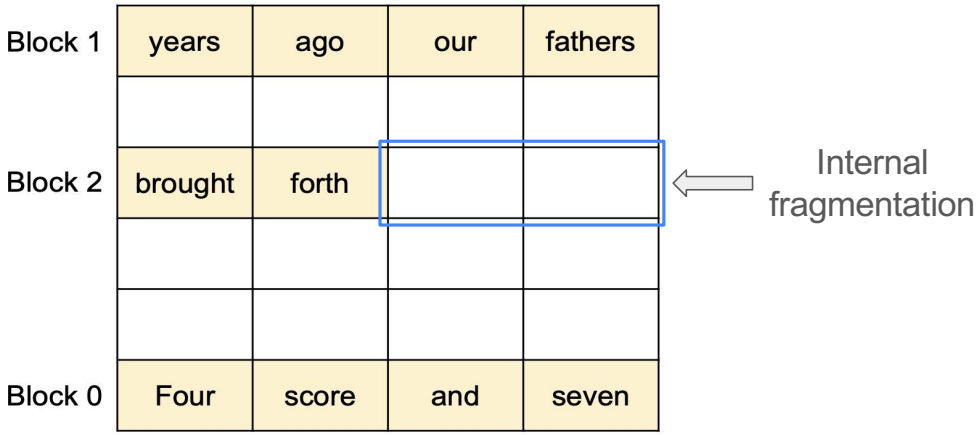


Block Table For Managing KV Cache Blocks



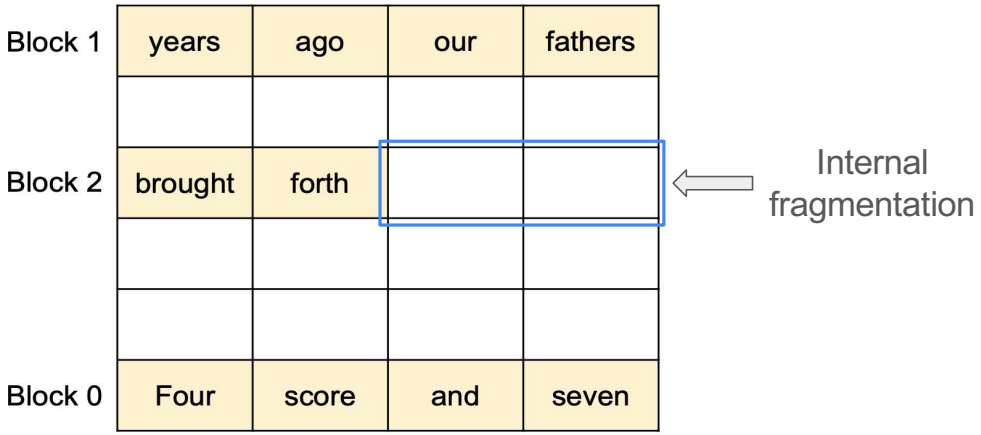
Memory Efficiency of PagedAttention

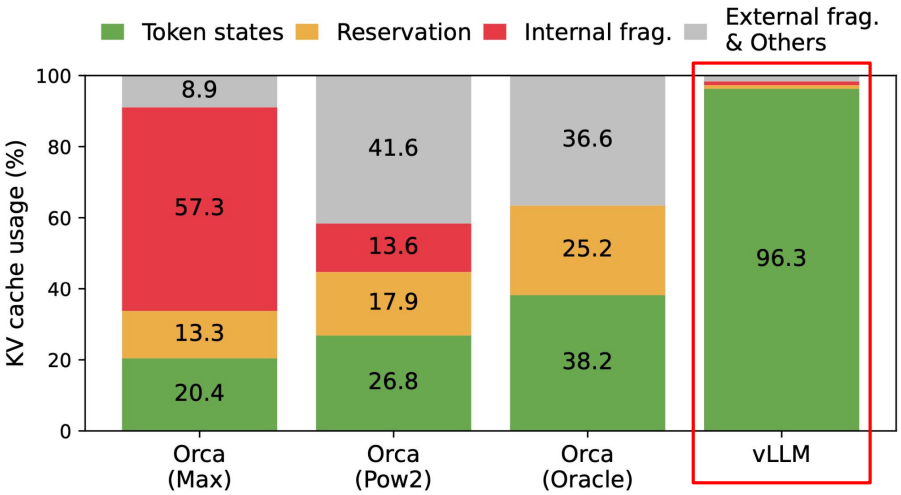
- Minimal internal fragmentation
  - # of wasted tokens per sequence < block size
- No external fragmentation



Memory Efficiency of PagedAttention

- Minimal internal fragmentation
  - # of wasted tokens per sequence < block size
- No external fragmentation

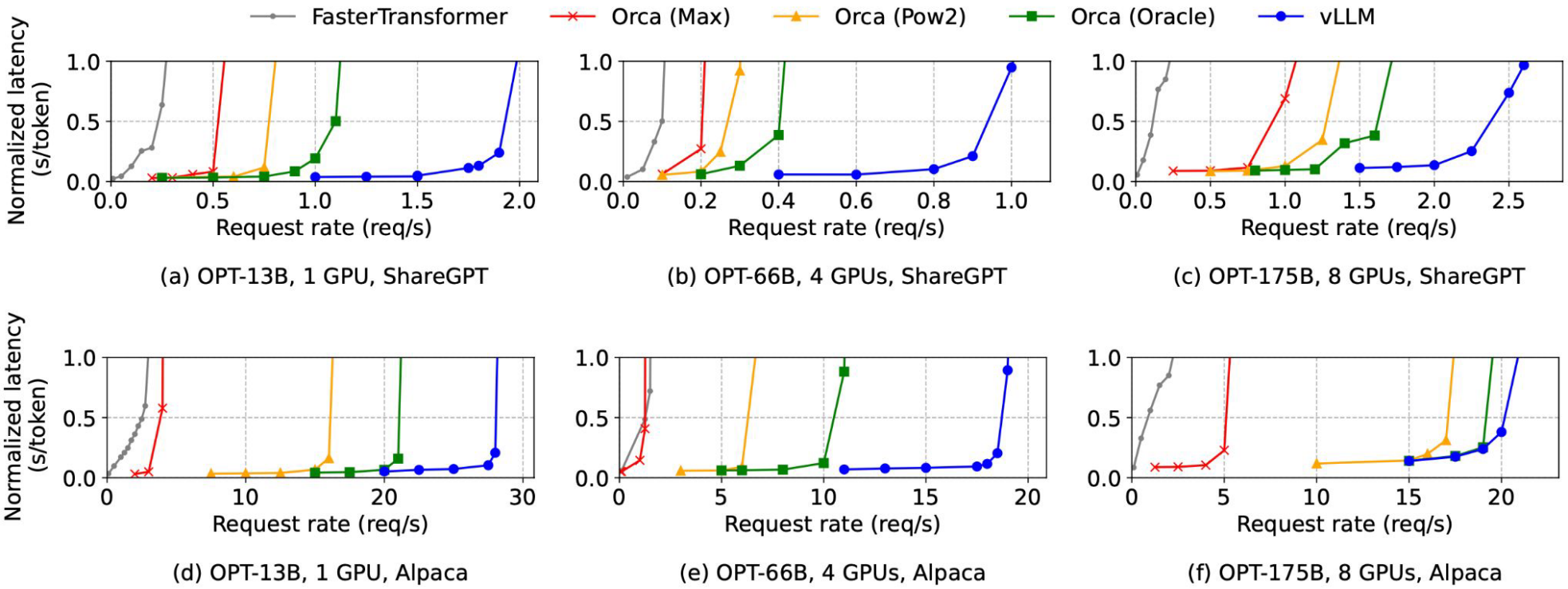




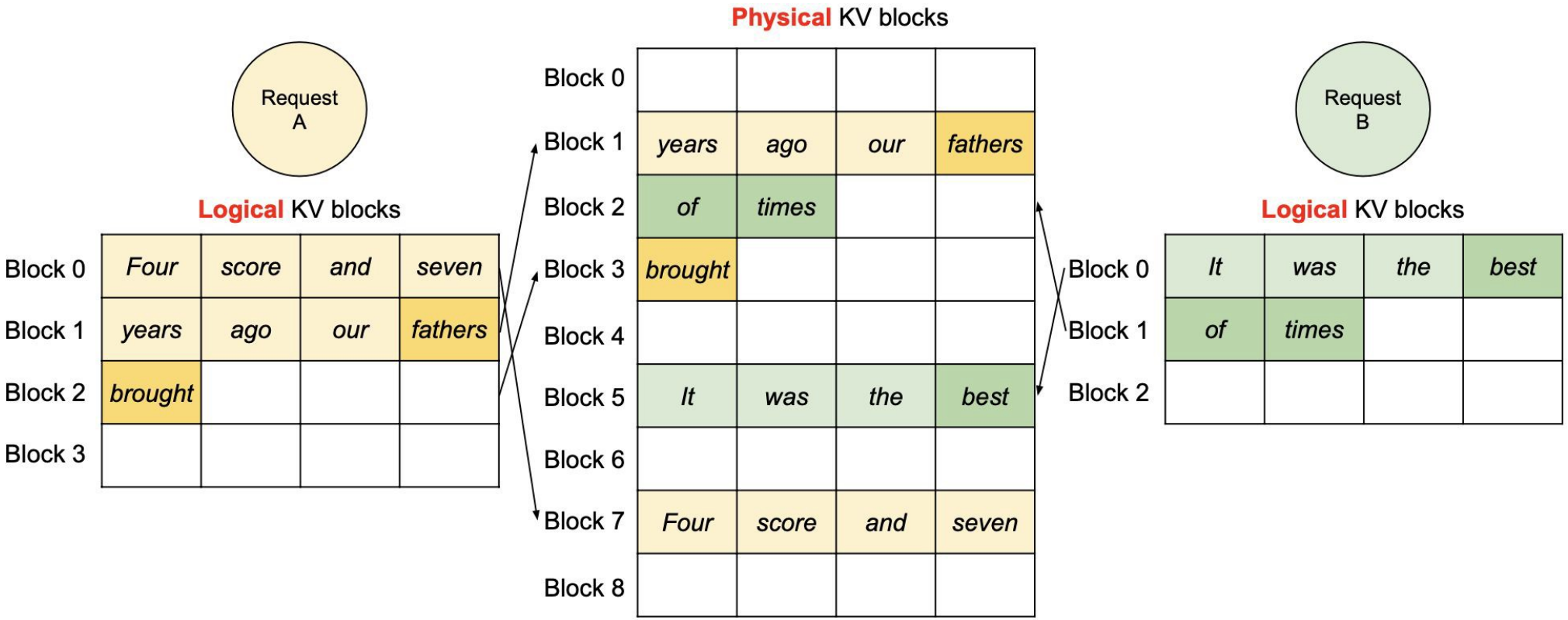
- With PagedAttention, wasted KV cache space is < 4% (3-5x improved memory utilization)

vLLM performance with basic generation

- one sample per request on three models and two datasets



Handling multiple requests

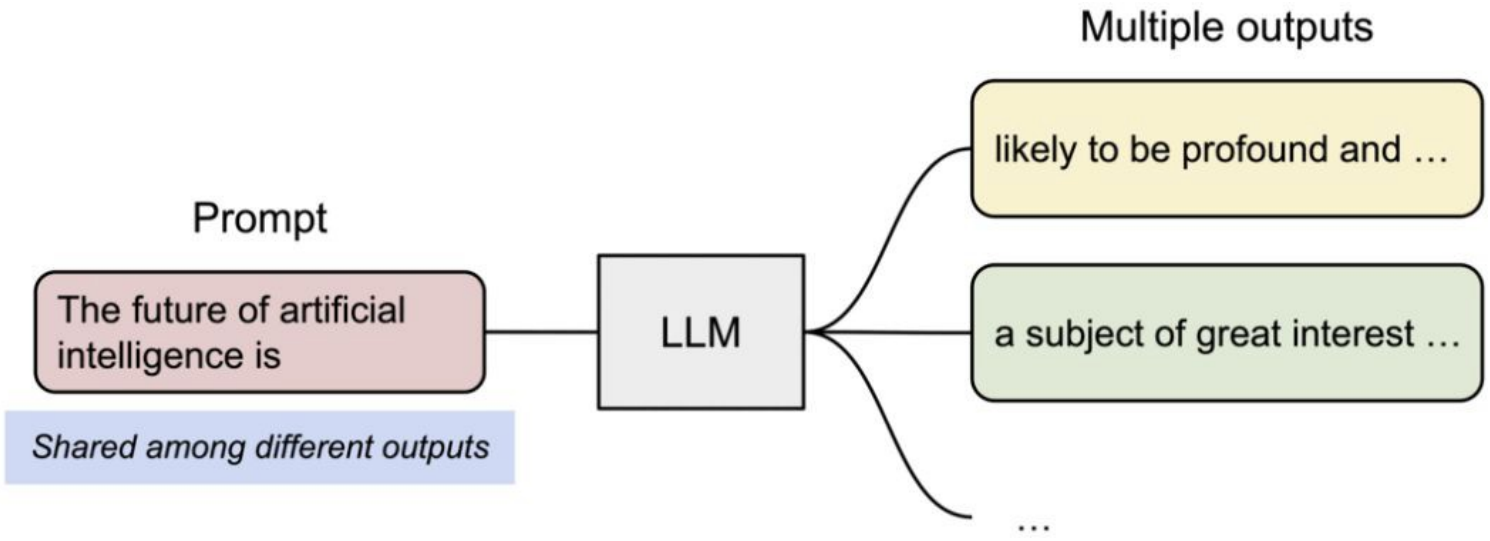


Application to Other Decoding Scenarios

Eg.

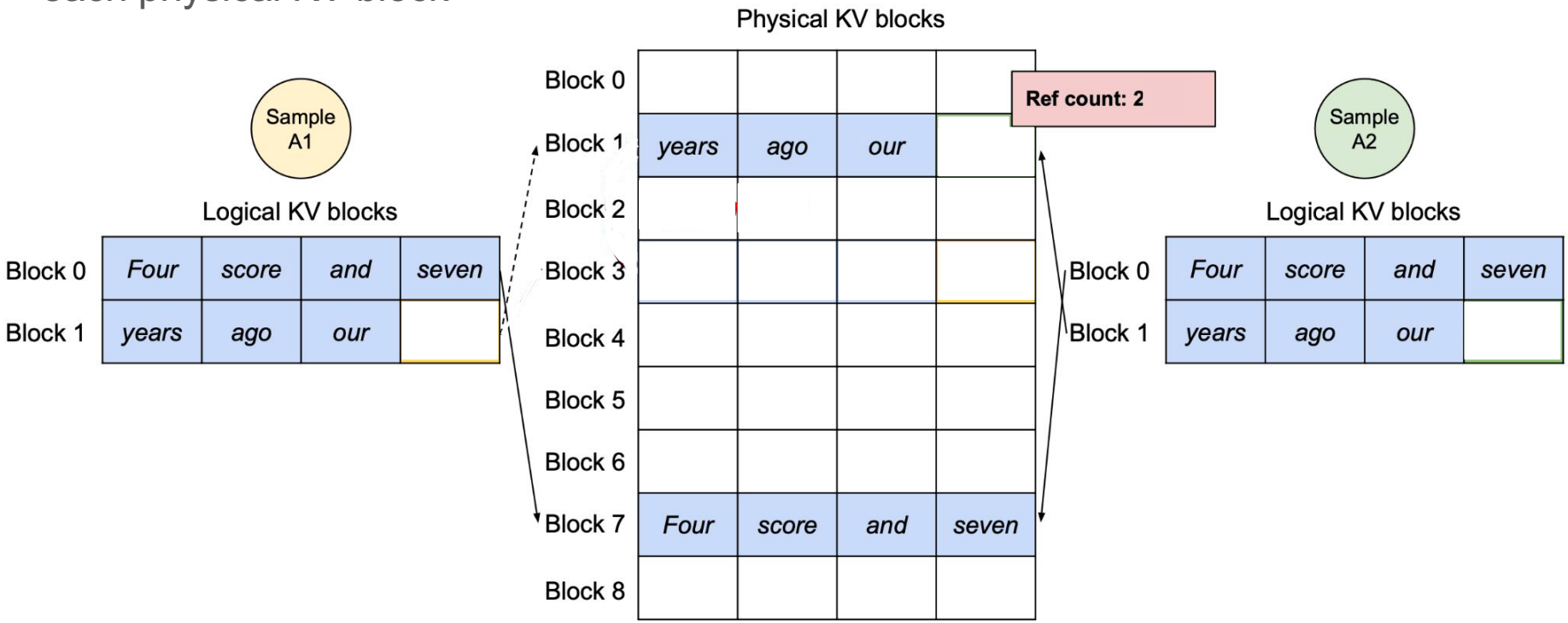
- Parallel sampling
- Beam search

Parallel Sampling



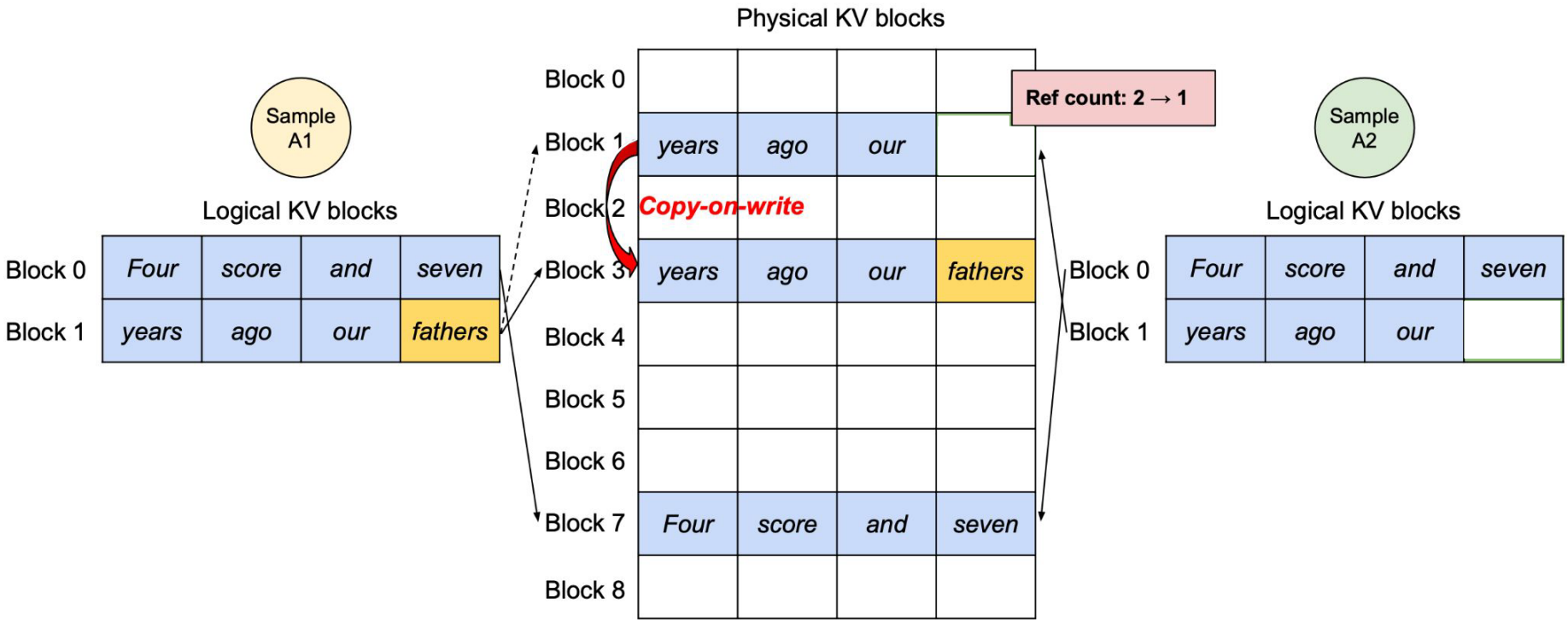
Parallel Sampling with PagedAttention

- Dynamic block mapping enables sharing of KV blocks
- Reference count keeps track of the number of logical KV blocks that are mapped to each physical KV block



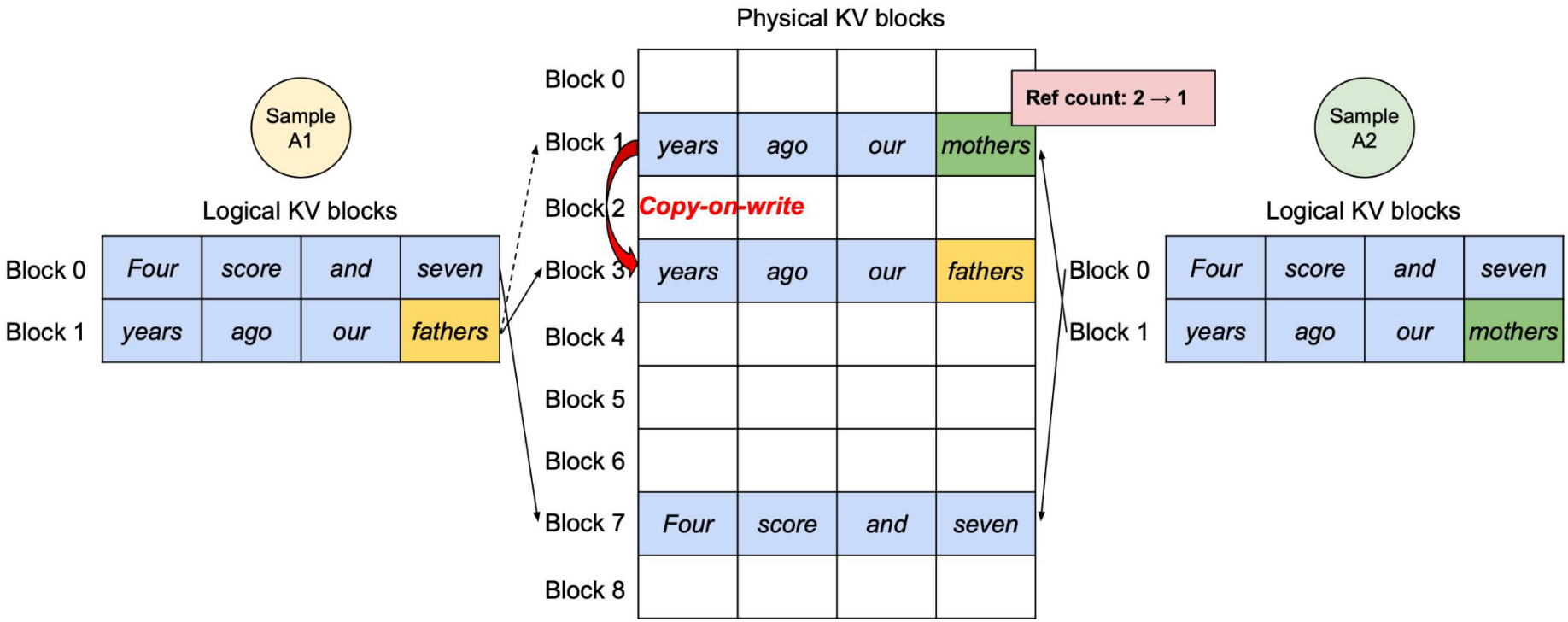
Parallel Sampling with PagedAttention

- At the generation phase, copy-on-write copies info to a newly allocated physical KV block when reference count > 1



Parallel Sampling with PagedAttention

- At the generation phase, copy-on-write copies info to a newly allocated physical KV block when reference count > 1



Limitations

- Choice of block size can have a substantial impact on the performance
- Increased kernel complexity and overhead
- Limited effectiveness in short-sequenced workloads
- Doesn’t natively support various models/GPU architectures

Main Takeaways

- Reduces memory fragmentation with paging
- Reduces memory usage with KV block sharing
- Enables batching of more requests, increasing the throughput of LLM inference

Table of contents

- Expose semantics to manage far memory better
  - **AIFM**: High-Performance Applications-Integrated Far Memory, **OSDI'20**
- Reinvent paging for new workloads:
  - Efficient Memory Management for Large Language Model Serving with **PagedAttention**, **SOSP'23**
- Predict accesses before demand misses happen
  - **PF-LLM**: Large Language Model Hinted Hardware Prefetching, **ASPLOS'26 (Best Paper)**

JC STEM Lab of
Future Advanced
Computing Technologies

香港科技大學
THE HONG KONG UNIVERSITY OF
SCIENCE AND TECHNOLOGY

PF-LLM: Large Language Model Hinted Hardware Prefetching

Entropy Xu
Research Assistant Professor
JC STEM FACT Lab, Director: Prof. Yuan Xie
The Hong Kong University of Science and Technology
March 25, 2026

Outline

1. Background: The Importance of Prefetching
2. Motivation:
    1. Online Learning vs. Offline Learning
    2. Why Do LLMs Work for Prefetching Hinting?
3. Method:
    1. LLM as a code semantic learner for classifying memory access patterns.
4. Results
5. Insights for Future CPU micro-architecture design.

Background

The Importance of Prefetching

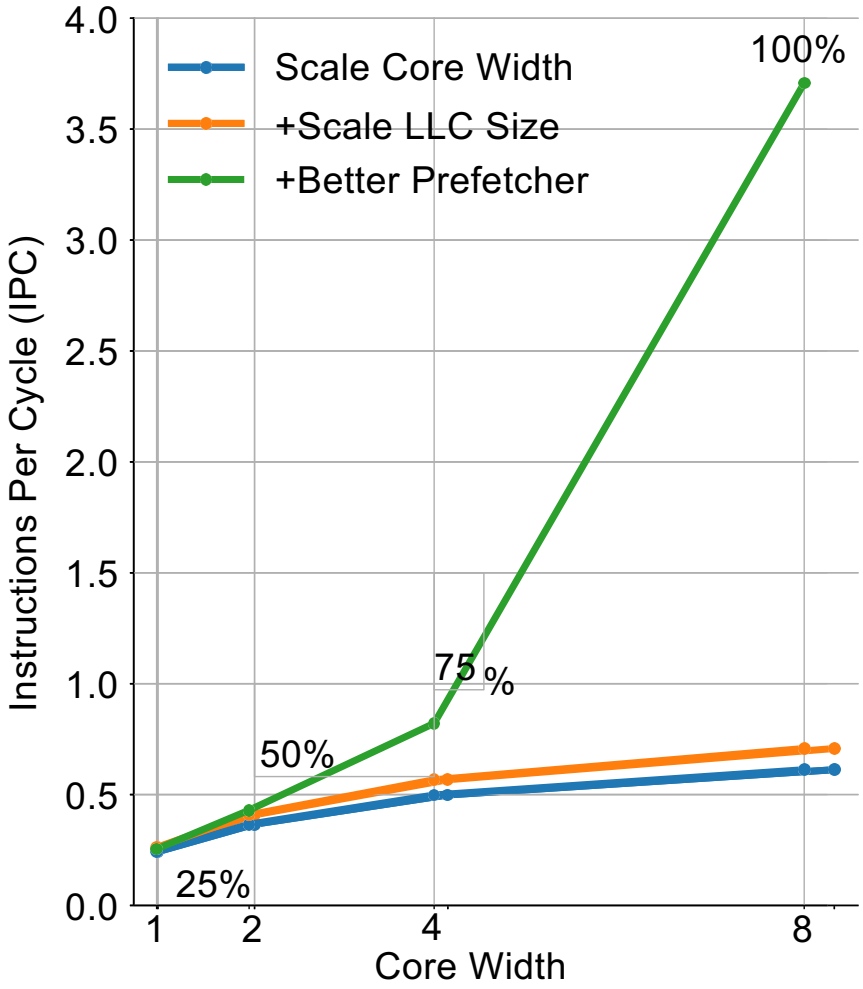
Prefetching is Effective

And Efficient

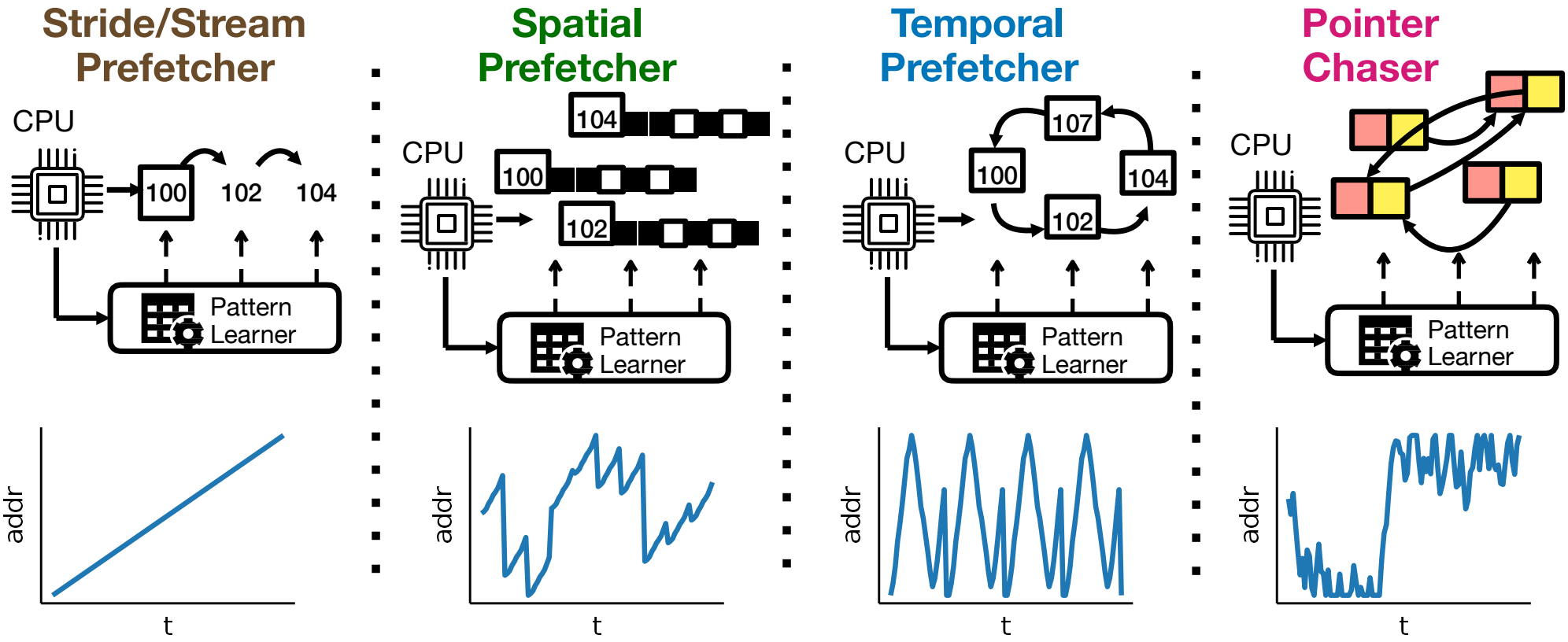
Prefetching
→
essential for single-core IPC

Beats spending die area on larger caches

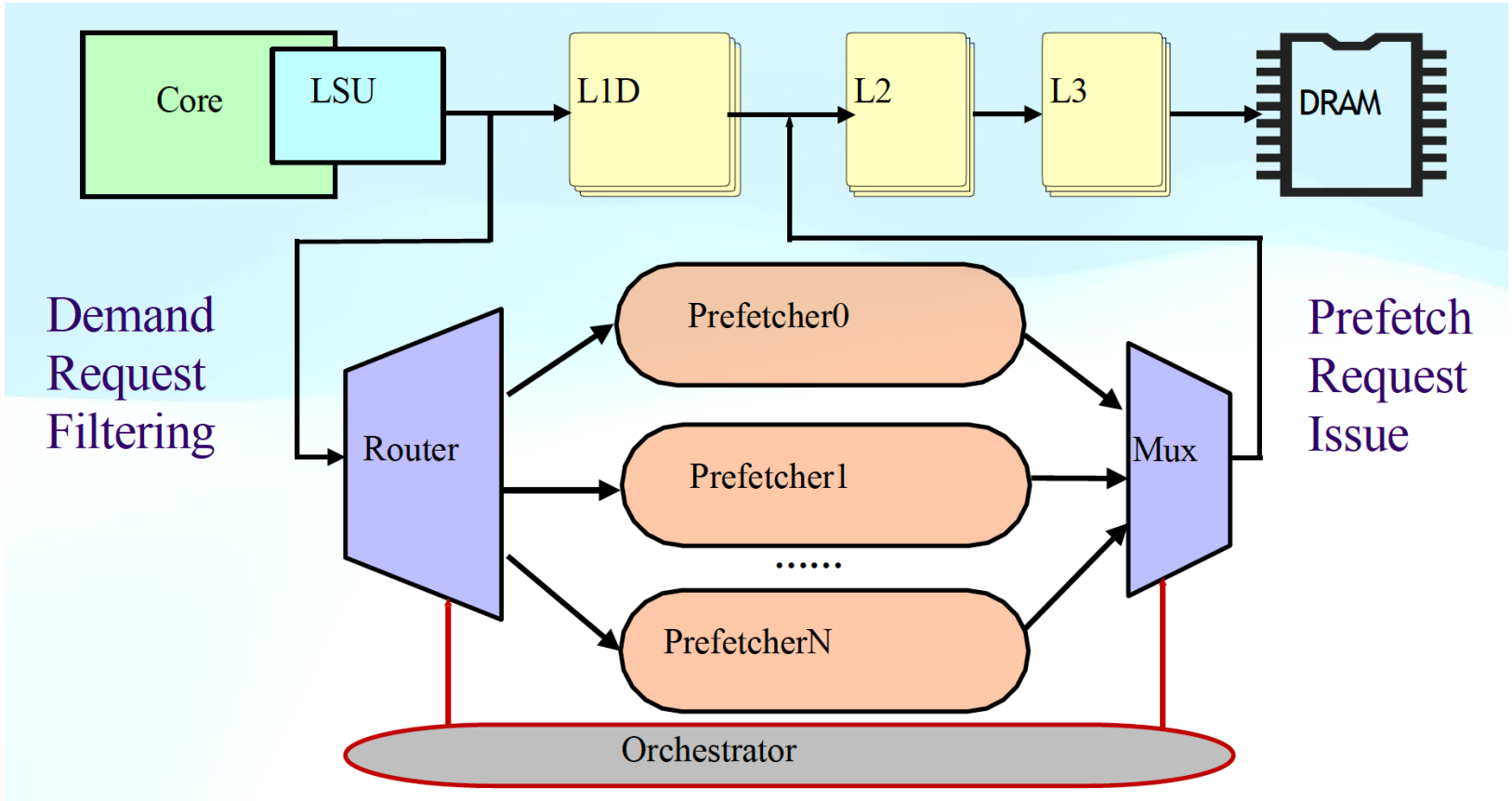
Enables wide-issue scaling



Hardware Prefetchers



Hardware Prefetcher Ensemble



Problems of Existing Prefetchers

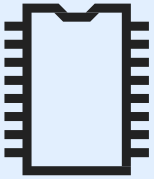
- Existing Prefetchers rely on **on-chip online learning**.

Slow Convergence

- Relies on online table-based heuristic to converge.
- Unstable and slow convergence for complex patterns.

Limited On-Chip Logic Budget

- Of course you cannot run an LLM at 3.0GHz on-chip.
- Table-based heuristics require large on-chip-storage.



Code Context Unaware

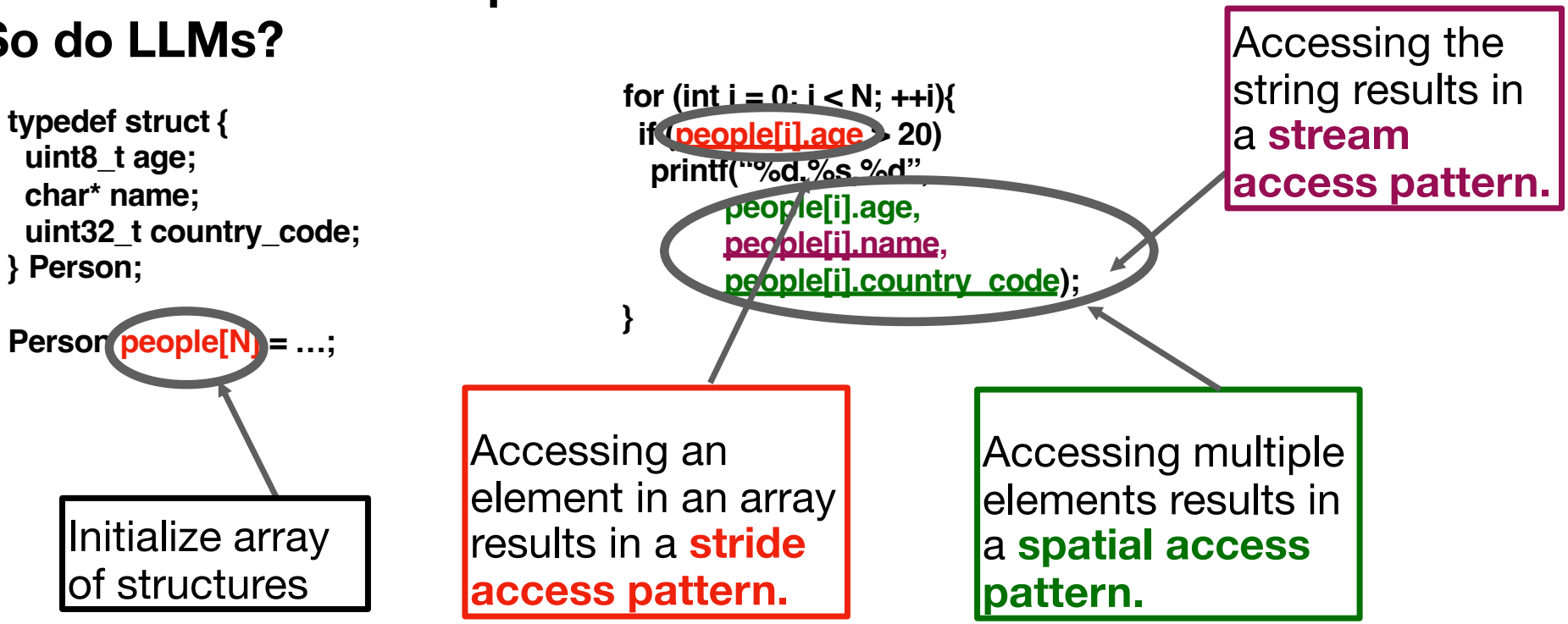
- Code context information remains unexploited for online decisions.

Motivation

Can we move the hard decision of
what, when and how to prefetch
from runtime hardware to an offline LLM-based analysis?

Human Developers Know How to Prefetch

So do LLMs?



Main Ideas

- **Offline**
- Fine-tune an LLM called PF-LLM to analyze code context and predict the
- **best prefetching policy** for each load instruction.
- Offline: expensive reasoning is allowed.
- Serves as an oracle for online decisions.

Online

- Use a lightweight hardware ensemble that **consumes** the **offline-generated** hints to perform **runtime decisions.**
- Runtime: hardware stays simple and low-latency.
- Achieve near-oracle guidance without online trial-and-error.

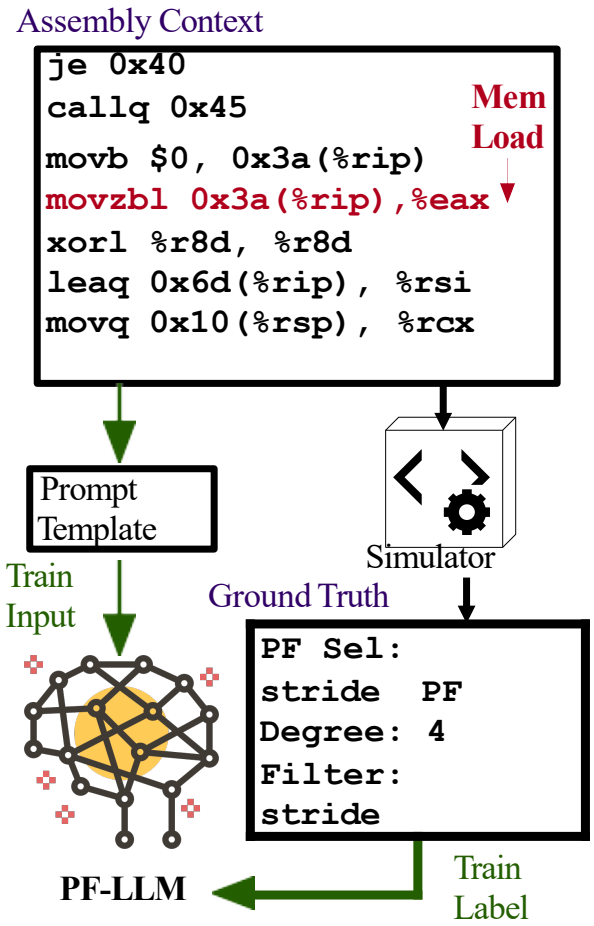
Method

Offline LLM to Hint Online Prefetching

Step 1: PF-LLM Model Training

Overview

- **Input:**
  - Assembly context formatted into a prompt (128 lines before and after)
  - Inference on each load instruction.
- **Output:**
  - Prefetch policy including:
    - Prefetcher **Selection**
    - Prefetcher Demand Request **Filter**
    - Prefetch **Degree** (aggressiveness)



Step 1: PF-LLM Model Training

Prompt Formatting

```
<|im_start|>system
You are a helpful assistant ...
<|im_end|>

<|im_start|>user
movzbl 0x3adef0(%rip), %eax
xorl %r8d, %r8d
leaq 0x6dc8d(%rip), %rsi
<load>movq 0x10(%rsp), %rcx</load>
movl 0x1c(%rsp), %edx
movq 0x3ab11d(%rip), %rdi
orl $2, %eax
<|im_end|>

<|im_start|>assistant \{
    "PF Sel": "stride",
    "PF Degree": 2,
    "Filter": "stream"
\}<|im_end|>
```

Generic System Prompt

Assembly code as input

Input (prefill)

Output (decode)

Prefetch Policy as Label

Why Bother?

- Leverage pre-trained code-comprehension knowledge in foundation models.

Step 1: PF-LLM Model Training

- Dataset Collection and Training

PC	PF Type	PF Degree	AMAT
0x1234	Stride	1	25.2
0x1234	Stride	2	28.3
0x1234	Stream	1	101.0
0x1234	Stream	2	103.9
0x1235	…	…	…
…	…	…	

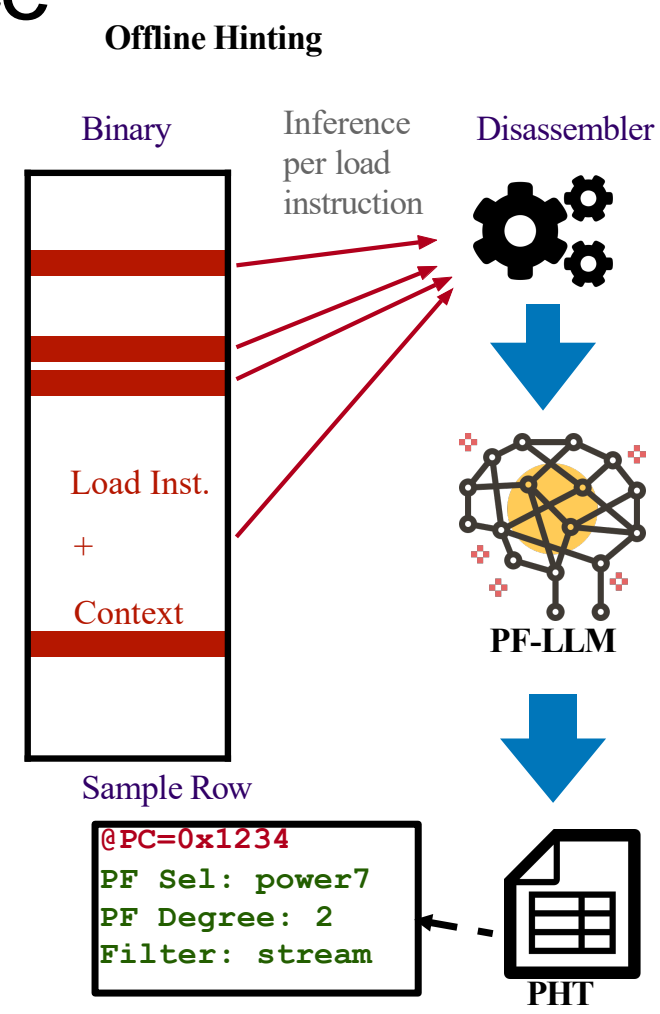
- For one benchmark in the training set, run simulations for all permutation of the prefetch policies.
- Modify ChampSim to record the AMAT of each PC with each policy.
- Select best policy and worst policy based on the AMAT.

PF Sel: stride
PF Degree: 1
Filter: stream

Step 2: PF-LLM Model Inference

Inference at compile time.

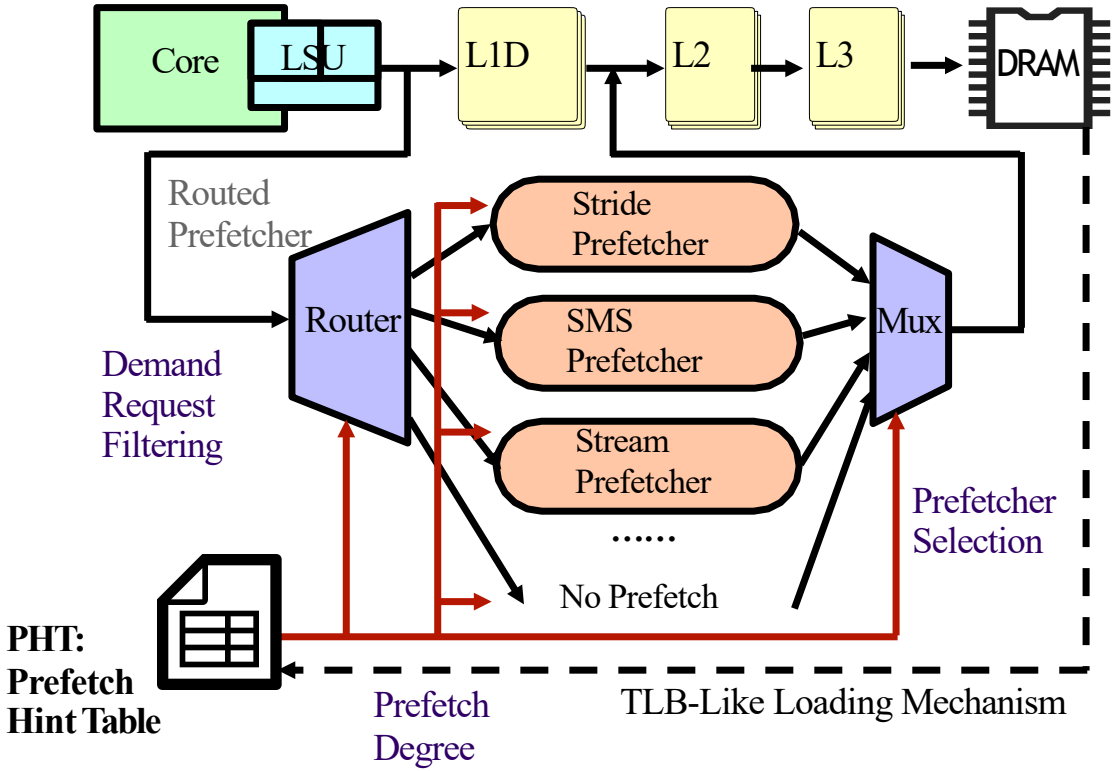
- The [Qwen-2.5-Coder-0.5B-Instruct] model is fine-tuned with our dataset.
- At compilation time, run **one model inference with the same prompt formatting for each load instruction** in the binary.
- The **per load-instruction hints** are stored in a tabular file called prefetch hint table for later use.



Step 3: Use LMHint Prefetcher at Runtime

Simple hardware but with educated HINTs.

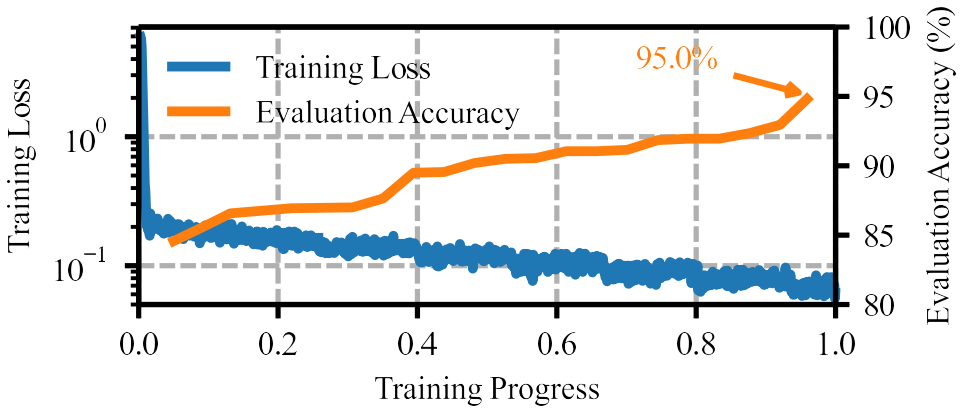
- Start with Prefetcher Ensemble Hardware
- Use a TLB-like Prefetch Hint Buffer to load hints.
- Route and Filter Prefetchers based on HINTs



Results

State-of-the-art IPC, Negligible On-chip Overhead

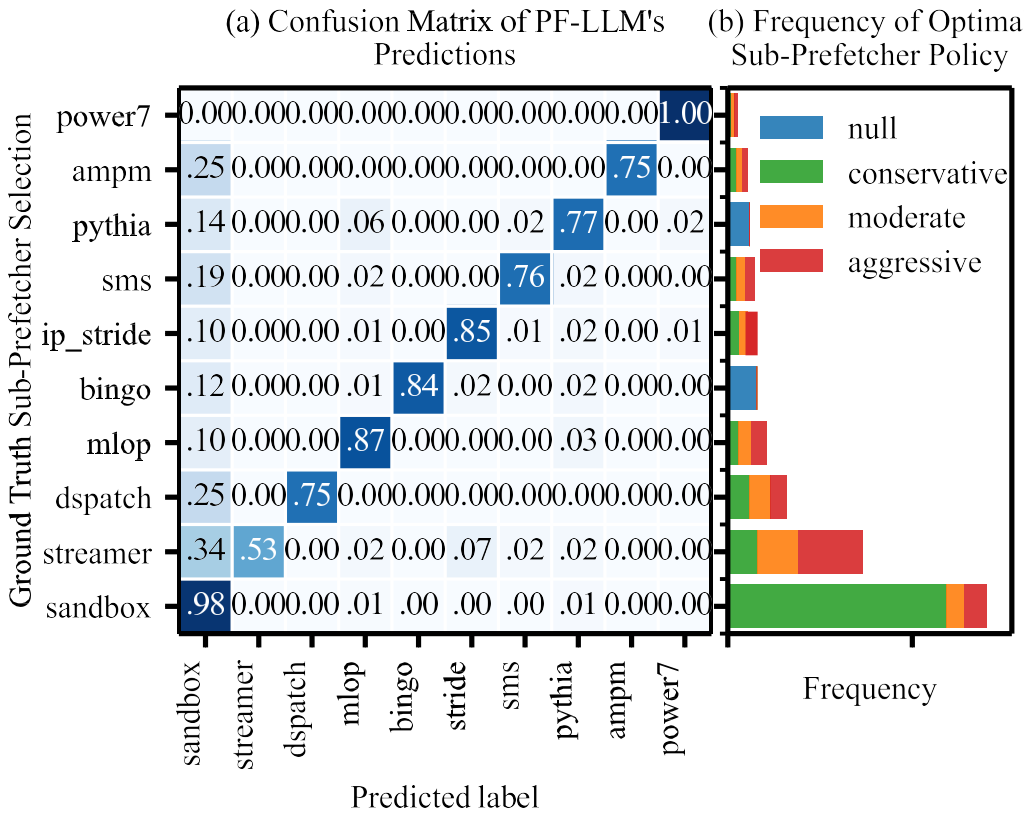
PF-LLM Training Results



Steady convergence

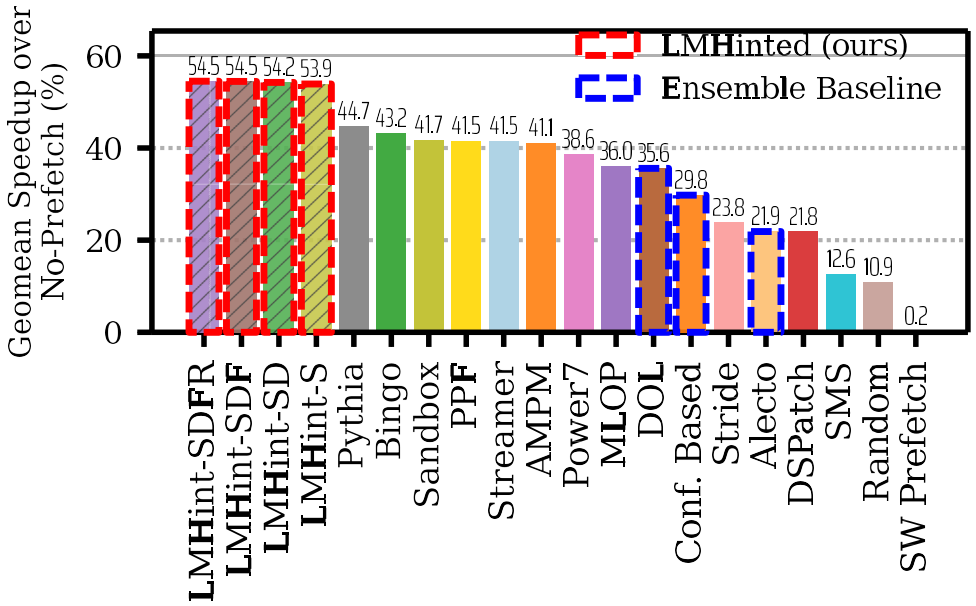
Strong diagonal line

95% all-match accuracy



Benchmark on SPEC 2017

- **S**: Select Policy only
- **SD**: + Degree
- **SDF**: + Filter
- **SDFR**: + Reduce to only four most-frequently used PF



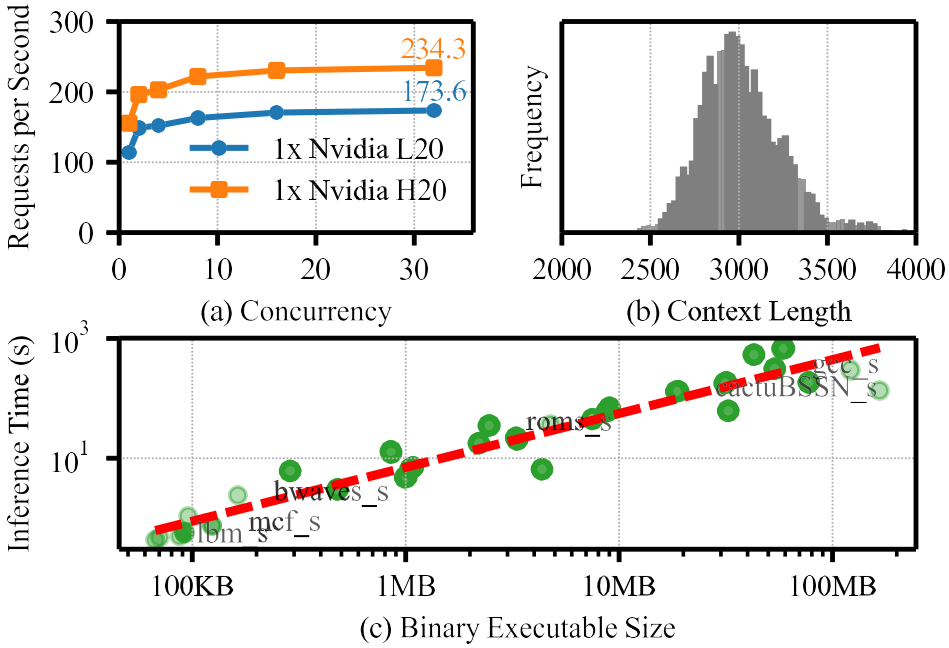
- +9.8% IPC over state-of-the-art single hardware prefetching baselines
- +18.9% IPC over prior ensemble methods

Overhead Analysis of PF-LLM

Small 0.5B model;
~3k-token short context length;
high-concurrency.

With vLLM Serving, Throughput is acceptable.

Hinting speed on 8xH20 system is on-par with a 16-core compilation system.



Insights

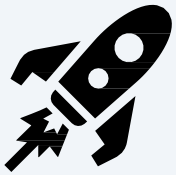
Offload hard online decision problems to offline LLM-generated Hints

Benefits of PF-LLM

Offline intelligence for code-aware policy + simple runtime hardware for fast execution.

Good Performance

- +9.8% IPC, roughly a full generation's worth of single-core IPC improvement.

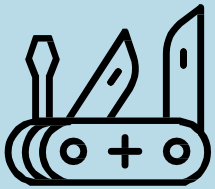


Offline Oracle

- Offline LLM instantly decides "when, how & how aggressively" to prefetch
- Eliminates slow online learning and runtime training cost

Works Anywhere

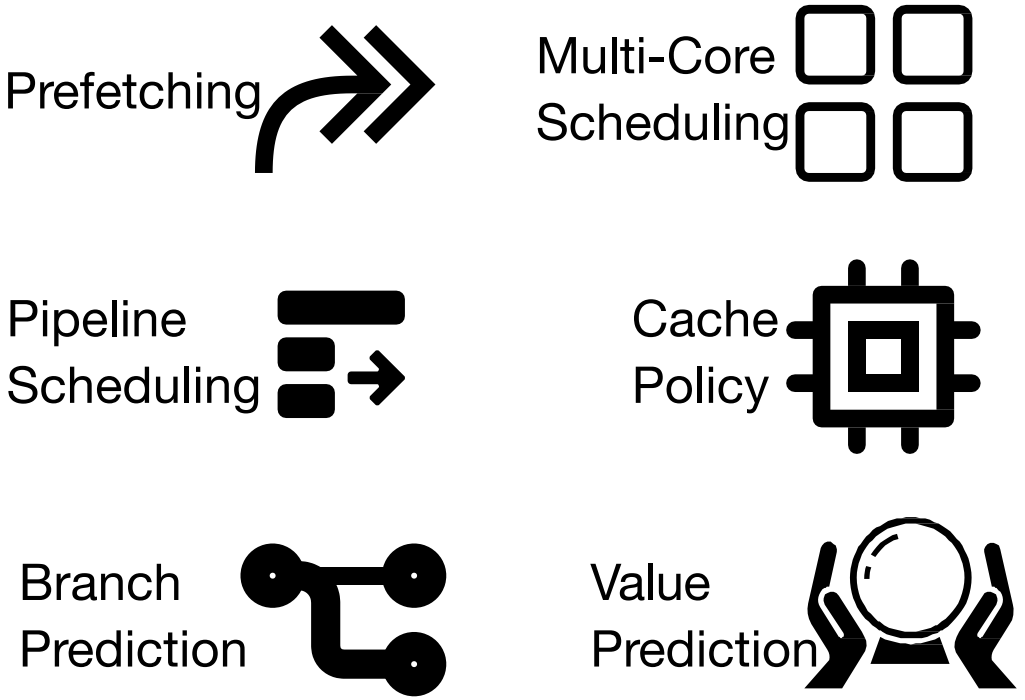
- The code context information remains un-exploited for online decisions.



LLM Hinted CPU micro-Architecture

Offline intelligence for code-aware policy + simple runtime hardware for fast execution.

- Prefetcher is just one of the important **micro-architectural decisions**.
- Using LLMs to gather offline code-context knowledge, apply to **online on-chip** policies.
- Towards a new paradigm of LLM-hinted CPU design.



Conclusion

- Memory management is no longer just OS paging within one machine.
  - Infiniswap: extend memory beyond local DRAM using remote memory, while keeping the classic VM abstraction.
  - AIFM: when paging is too coarse, expose application semantics and manage memory at object granularity.
  - PagedAttention: modern workloads like LLM serving reinvent memory management — from segmentation-like contiguous allocation to paging-like block allocation with indirection.
  - PF-LLM: memory systems are becoming predictive, using learned hints to improve prefetching and reduce stalls.
- Big picture: the trend is from reactive, transparent page movement to proactive, semantics-aware, and workload-aware memory orchestration.

Reading for next lecture

- Operating System Concepts:
  - Chapter 11.1 & 12.1-12.2 & 13.1

Or

- Operating System: Three Easy Pieces:
  - Chapters 35-39

Thanks